Del modelo a la máquina: simulación con SimGrid y emulación reproducible con Vagrant de estrategias de planificación y balanceo de carga

Sofía Quintana, Facundo Salazar y Romina Vera

Facultad de Ingeniería, Tecnología y Arquitectura, Universidad de la Cuenca del Plata

Ingeniería en Sistemas de Información · Sistemas Operativos Distribuidos

Trabajo Integrador Final (AE4, Propuesta B) · Comisión A · Sede Posadas

Docentes: Juan Emiliano Jara y Paola Quiñones

Noviembre de 2026

Resumen

Construimos un laboratorio distribuido reproducible para decidir qué estrategia de planificación conviene en una red de procesamiento de datos agrometeorológicos con computadoras heterogéneas. El laboratorio combina emulación (un coordinador y tres trabajadores en máquinas virtuales definidas con Vagrant y aprovisionadas con Ansible, con latencia impuesta con netem y velocidad de CPU con *cgroups*) y simulación con SimGrid. Comparamos Round Robin estático, Menos cargado centralizado y Robo de trabajo, escritos en un único módulo que usan tanto el clúster como el simulador. En 90 corridas reales, con tres niveles de latencia y dos de heterogeneidad a igual capacidad total, las estrategias no se diferencian en tiempo total cuando los nodos son iguales, pero con nodos desiguales el robo de trabajo termina en 38 a 40 s contra 42 a 43 s de Menos cargado y 56 a 57 s de Round Robin ($p < 0{,}001$). Calibrado con mediciones del clúster, el simulador reproduce el makespan con un error medio de 1,1 % y conserva el orden de las estrategias en todas las condiciones, aunque es menos preciso en el tiempo de espera. Extrapolado hasta 768 nodos, predice que el límite del diseño centralizado no es la CPU del coordinador sino su enlace de red, por el que pasan los datos de todas las tareas y que se satura cerca de los 490 trabajadores. Concluimos que la simulación calibrada y validada es confiable para comparar estrategias y para ubicar cuellos de botella, no para predecir valores absolutos de métricas finas fuera del rango validado.

Palabras clave: sistemas distribuidos, balanceo de carga, robo de trabajo, SimGrid, Vagrant, infraestructura como código.

Índice

Resumen 2

Introducción 6

Los sistemas operativos distribuidos y su lugar en la infraestructura de TI 6

El problema de la gestión de recursos y la planificación global 7

Tres formas de estudiar un sistema distribuido 7

La reproducibilidad como requisito científico y profesional 8

Objetivos 9

Objetivo general 9

Objetivos específicos 9

Investigación teórica 10

Planificación y balanceo de carga 10

SimGrid 12

Vagrant 14

Ansible 16

Middleware de comunicación 16

Emulación de red con tc y netem 17

Monitoreo con Prometheus y Grafana 18

Evaluación de desempeño 18

Organización del equipo 19

Diagnóstico de experiencia 19

Roles y distribución de tareas 19

Materiales y métodos 21
Arquitectura del laboratorio 21
Software y hardware 23
Carga de trabajo 23
Estrategias implementadas 24
Diseño experimental 25
Modelo de simulación 26
Implementación práctica 28
Paso 0: diagnóstico y acta de roles 28
Paso 1: clúster como código con Vagrant 28
Paso 2: aprovisionamiento con Ansible 30
Paso 3: implementación de las estrategias 31
Paso 4: modelo en SimGrid 32
Paso 5: experimentación en el clúster real 33
Paso 6: calibración y validación 34
Paso 7: extrapolación a gran escala 36
Paso 8: prueba de reproducibilidad 36
Análisis de resultados y discusión 38
Desempeño y comparación de las estrategias 38
Efecto de la heterogeneidad y de la latencia 41
Validez del simulador 42

Escalabilidad 45

Consumo de recursos 49

Ventajas y desafíos 50

Conclusión 51

Hallazgos sobre planificación y balanceo de carga 51

Respuesta a la pregunta guía 52

Reflexión de cada integrante desde su rol 53

Propuestas para trabajos futuros 54

Referencias 56

Anexo A: bitácoras individuales 59

Bitácora individual - Salazar, Facundo 59

Bitácora individual - Quintana, Sofía 62

Bitácora individual - Vera, Romina 66

Anexo B: acta de roles 71

Anexo C: planilla de corridas 74

Anexo D: archivos de plataforma y comandos de reproducción 74

Del modelo a la máquina: simulación con SimGrid y emulación reproducible con Vagrant de estrategias de planificación y balanceo de carga

Introducción

Los sistemas operativos distribuidos y su lugar en la infraestructura de TI

Un sistema distribuido es una colección de computadoras independientes que se presenta ante sus usuarios como un único sistema coherente (Tanenbaum & van Steen, 2023). Coulouris et al. (2012) lo definen desde el mecanismo: componentes ubicados en computadoras conectadas en red que se comunican y coordinan sus acciones únicamente mediante el intercambio de mensajes. Las dos definiciones marcan la tensión que atraviesa toda la materia: hacia afuera se busca transparencia (de ubicación, de fallas, de concurrencia) y hacia adentro no hay memoria compartida ni reloj global, solo mensajes que tardan y que pueden perderse.

Históricamente, la idea de un *sistema operativo* distribuido, que gestionara los recursos de varias máquinas como si fueran una sola, se exploró en los años ochenta con proyectos de investigación como Amoeba, Mach, V o Sprite (Coulouris et al., 2012; Tanenbaum & van Steen, 2023). Esos sistemas no llegaron a imponerse de manera masiva, pero sus ideas (micronúcleos, comunicación por mensajes, migración de procesos, sistemas de archivos distribuidos) pasaron a los sistemas operativos de red y, sobre todo, a la capa de *middleware*. Con los clústeres de computadoras de bajo costo de los años noventa, las grillas de cómputo de los dos mil y la nube actual, el problema central se desplazó del núcleo del sistema operativo a los planificadores y orquestadores que reparten trabajo entre miles de nodos (Burns et al., 2016).

Hoy la infraestructura de TI de cualquier organización mediana es distribuida: servicios replicados, bases de datos en varios nodos, colas de mensajes, plataformas de procesamiento de datos y dispositivos en el borde de la red. El escenario de este trabajo es un

ejemplo cercano: una red de estaciones agrometeorológicas de la región chaqueña que envía periódicamente lotes de datos (temperatura, humedad, precipitación e imágenes de cultivo) que deben procesarse en un conjunto heterogéneo de computadoras, algunas nuevas y otras antiguas, unidas por enlaces de calidad dispar.

El problema de la gestión de recursos y la planificación global

La gestión de recursos en un sistema distribuido (Eje 2 de la materia) consiste en decidir qué nodo ejecuta cada unidad de trabajo, cuándo y con qué información. Sinha (1997) distingue tres enfoques: la asignación de tareas (decidir de antemano dónde corre cada tarea), el balanceo de carga (igualar la carga entre nodos) y el reparto de carga (*load sharing*, evitar solamente que haya nodos ociosos mientras otros tienen trabajo en espera). Encontrar la asignación óptima es un problema NP-difícil en el caso general, de modo que en la práctica se usan heurísticas (Sinha, 1997).

La heterogeneidad lo complica. Si todos los nodos son iguales, repartir la misma cantidad de tareas a cada uno es razonable; si un nodo tiene un tercio de la velocidad de otro, ese mismo reparto lo transforma en el cuello de botella. Además, el planificador nunca conoce el estado exacto de los demás nodos: lo que sabe le llegó por mensajes que viajaron por la red y que, cuando llegan, ya describen el pasado (Singhal & Shivaratri, 1994). Cuanto mayor es la latencia, más vieja es esa información y más mensajes de control hacen falta para mantenerla al día.

Tres formas de estudiar un sistema distribuido

Jain (1991) identifica tres técnicas para evaluar el desempeño de un sistema: el modelado analítico, la simulación y la medición sobre el sistema real o un prototipo. En sistemas distribuidos la medición suele hacerse sobre una emulación: un sistema real a escala reducida en el que algunas condiciones, como la latencia de la red o la velocidad de los nodos, se imponen artificialmente. La Tabla 1 resume sus ventajas y límites.

Tabla 1

Comparación de las técnicas para estudiar un sistema distribuido

Criterio	Modelado analítico	Simulación	Experimentación / emulación
Qué se obtiene	Fórmulas cerradas (teoría de colas)	Ejecución de un modelo en un reloj simulado	Mediciones sobre el sistema real o un prototipo
Costo y tiempo	Muy bajo	Bajo a medio	Alto (equipos, horas de corrida)
Escala alcanzable	Ilimitada	Miles a millones de nodos	La que se pueda montar
Realismo	Bajo: supuestos fuertes	Depende del modelo y su calibración	Alto, con ruido del entorno
Control y repetibilidad	Total	Total (determinista)	Parcial: interferencias, variabilidad
Principal riesgo	Simplificar de más	Confiar en un modelo no validado	Medir mal o sacar conclusiones de pocas corridas

Nota. Elaboración propia a partir de Jain (1991, cap. 3) y Casanova et al. (2014).

Las tres técnicas se complementan. Jain (1991) recomienda no confiar en el resultado de una sola hasta haberlo contrastado con otra, y ese es el recorrido que propone este trabajo: medir en un clúster pequeño y emulado, usar esas mediciones para calibrar y validar un simulador y, solo después, usar el simulador para extrapolar a escalas que no podemos montar.

La reproducibilidad como requisito científico y profesional

Un experimento que solo puede repetir quien lo hizo, en su computadora y recordando los pasos de memoria, no es verificable. En sistemas distribuidos el problema es agudo porque el resultado depende del entorno: versiones de paquetes, configuración de red, recursos asignados a cada máquina. La infraestructura como código (*Infrastructure as Code*) aborda ese problema describiendo la infraestructura en archivos de texto versionados que una herramienta convierte en máquinas, redes y servicios, de forma repetible y con el mismo resultado cada vez (Morris, 2020).

En este trabajo todo el laboratorio está definido así: un `Vagrantfile` crea las máquinas virtuales, los *playbooks* de Ansible las configuran, un único archivo de parámetros

fija la carga y el diseño experimental, la carga se genera con semillas fijas y los datos crudos se guardan como CSV en el mismo repositorio. Un tercero puede clonar el repositorio, ejecutar `vagrant up` y repetir el experimento.

Objetivos

Objetivo general

Construir un laboratorio distribuido reproducible que combine simulación (SimGrid) y emulación (máquinas virtuales definidas como código con Vagrant y aprovisionadas con Ansible) para comparar estrategias de planificación y balanceo de carga en una plataforma de procesamiento distribuido, validar el simulador contra mediciones reales y extrapolar el comportamiento a escalas que no pueden montarse en el laboratorio.

Objetivos específicos

1. Investigar y comparar herramientas de simulación (SimGrid) y de aprovisionamiento reproducible de infraestructura (Vagrant y Ansible).
2. Implementar tres estrategias de planificación y balanceo de carga (Round Robin estático, Menos cargado centralizado y Robo de trabajo) con un único módulo de decisión que usen tanto la simulación como el clúster real.
3. Diseñar un experimento controlado con factores, niveles, métricas y repeticiones definidos de antemano.
4. Calibrar y validar el modelo de simulación contra mediciones reales, cuantificando el error relativo.
5. Extrapolar mediante simulación a plataformas de 16 a 512 nodos y discutir la validez de esas predicciones.

Investigación teórica

Planificación y balanceo de carga

Planificación estática y dinámica, centralizada y distribuida

Una política de planificación es **estática** cuando la decisión de dónde ejecutar cada tarea se toma sin mirar el estado actual del sistema, a partir de información conocida de antemano o de una regla fija; es **dinámica** cuando usa información del estado en el momento de decidir (Sinha, 1997). Las estáticas son simples y no cuestan mensajes, pero no reaccionan a lo que pasa; las dinámicas se adaptan a cambio de recolectar información, lo que consume mensajes, tiempo de CPU y, sobre todo, tiempo de red.

La segunda dimensión es quién decide. En una política **centralizada** un único nodo (el coordinador) concentra la información y toma todas las decisiones: es fácil de implementar y razonar, pero ese nodo es un punto único de falla y un posible cuello de botella cuando crece la cantidad de nodos o de tareas. En una política **distribuida** cada nodo decide con información parcial, lo que escala mejor pero hace más difícil garantizar buenas decisiones globales (Singhal & Shivaratri, 1994; Sinha, 1997).

Componentes de un algoritmo de distribución de carga

Shivaratri et al. (1992) descomponen cualquier algoritmo dinámico de distribución de carga en cuatro políticas:

- **Transferencia:** decide cuándo un nodo debe enviar o pedir trabajo, normalmente con un umbral sobre la longitud de su cola.
- **Selección:** decide qué tarea se transfiere; conviene elegir tareas que todavía no empezaron, para no pagar el costo de migrar un proceso en ejecución.
- **Ubicación:** decide con qué nodo se hace la transferencia: al azar, sondeando algunos nodos o consultando información global.
- **Información:** decide qué información de estado se recolecta, cuándo y de dónde.

También distinguen entre algoritmos **iniciados por el emisor**, en los que un nodo sobrecargado busca a quién darle trabajo, e **iniciados por el receptor**, en los que un nodo ocioso sale a buscar trabajo. Los primeros funcionan bien con carga baja o media, pero con carga alta sus sondeos casi siempre fracasan y empeoran la situación; los segundos se comportan al revés (Shivaratri et al., 1992). Eager et al. (1986) mostraron, además, que políticas muy simples, con poca información de estado, obtienen la mayor parte del beneficio que podrían dar políticas complejas.

Estrategias evaluadas

Round Robin estático. La tarea i va al nodo $i \bmod N$. No usa información de estado ni mensajes de control. Reparte la misma *cantidad* de tareas a cada nodo, lo que solo equivale a repartir la misma *carga* si los nodos son iguales y las tareas cuestan parecido.

Menos cargado, centralizado en el coordinador. El coordinador lleva la cuenta de las tareas que despachó a cada trabajador y todavía no fueron confirmadas, y asigna cada tarea nueva al que tiene menos pendientes; en teoría de colas equivale a "unirse a la cola más corta". Es dinámica y centralizada. No necesita mensajes extra porque usa las confirmaciones que los trabajadores envían igual al terminar cada tarea, pero cada decisión recorre los N trabajadores y la información puede estar desactualizada en el tiempo que tarda una confirmación en llegar.

Robo de trabajo. Blumofe y Leiserson (1999) analizaron un planificador en el que cada procesador ejecuta su propia cola de trabajo y, cuando se queda sin tareas, se convierte en ladrón: elige una víctima al azar y le roba trabajo. Demostraron que, para computaciones multihilo totalmente estrictas, el tiempo esperado de ejecución con P procesadores es $T_1/P + O(T\infty)$ (T_1 es el trabajo total y $T\infty$ la longitud del camino crítico) y que la comunicación también está acotada. Es una política distribuida, dinámica e iniciada por el receptor: los

nodos ocupados no gastan nada en balancear y el costo de comunicación recae en los ociosos, que no tenían nada mejor que hacer.

El poder de dos opciones. Mitzenmacher (2001) estudió el modelo "supermercado": si cada tarea elige al azar *d* servidores y se une al de cola más corta, pasar de $d = 1$ a $d = 2$ reduce exponencialmente la cola máxima esperada; agregar más opciones mejora poco. Es la base de planificadores distribuidos como Sparrow (Ousterhout et al., 2013). La consigna permitía elegir entre esta estrategia y el robo de trabajo; elegimos el robo de trabajo porque contrasta mejor con la estrategia centralizada (decisión distribuida e iniciada por el receptor, con comunicación directa entre trabajadores) y porque su beneficio frente a la heterogeneidad es el que más interesa en el escenario. La Tabla 2 ubica las tres estrategias en la clasificación anterior.

Tabla 2

Las tres estrategias en la clasificación de Shivaratri et al. (1992)

Dimensión	Round Robin estático	Menos cargado	Robo de trabajo
Tipo	Estática	Dinámica	Dinámica
Quién decide	Coordinador	Coordinador (centralizada)	Cada trabajador (distribuida)
Iniciada por	—	Emisor (al llegar la tarea)	Receptor (nodo ocioso)
Transferencia	Siempre, al llegar	Siempre, al llegar	Cuando la cola local queda vacía
Selección	La tarea que llega	La tarea que llega	Mitad de la cola de la víctima, desde el final
Ubicación	Siguiente en orden cíclico	Trabajador con menos pendientes	Víctima al azar
Información	Ninguna	Pendientes por trabajador (de las confirmaciones)	Solo la respuesta de la víctima
Mensajes extra por tarea	0	0	Pedido y respuesta de robo

SimGrid

SimGrid es un marco de simulación de aplicaciones distribuidas desarrollado desde 1999 por un consorcio de grupos de investigación, en su mayoría franceses (Casanova et al., 2014; Legrand et al., 2003). Su objetivo es estudiar el comportamiento de una aplicación

sobre una plataforma (computadoras, enlaces, topología) sin necesidad de tener esa plataforma, y obtener resultados en poco tiempo y en forma determinista.

Arquitectura e interfaz S4U. La aplicación se escribe como un conjunto de **actores**: funciones que corren en un *host* simulado y que ejecutan cómputo (`execute`), se comunican por **buzones** (*mailboxes*, con `put` y `get`) y se sincronizan con mutex, semáforos o barreras. S4U es la interfaz actual, disponible en C++ y en Python (SimGrid Team, s.f.). El código del actor corre de verdad, pero el tiempo que transcurre no es el del reloj de la computadora: el núcleo de SimGrid calcula cuánto tarda cada actividad según los recursos que comparte con las demás y avanza un reloj simulado.

Modelos de CPU y red. Un *host* tiene una velocidad (operaciones por segundo) y una tarea de *C* operaciones tarda *C* dividido la porción de velocidad que le toca. La red no se simula paquete por paquete, como en ns-3, sino con **modelos de flujo**: cada comunicación es un flujo que atraviesa enlaces con ancho de banda y latencia, y el ancho de banda de cada enlace se reparte entre los flujos que lo comparten resolviendo un problema de equidad max-min (Casanova et al., 2014). El modelo por defecto (LV08) agrega factores de corrección calibrados para transferencias TCP grandes; el modelo CM02 calcula el tiempo de un mensaje solo como latencia más tamaño dividido ancho de banda. Velho et al. (2013) estudiaron la validez de estos modelos frente a simulaciones a nivel de paquete y mostraron que son precisos para transferencias grandes y estables, y que se apartan de la realidad con mensajes chicos o cuando hay mucha contención.

Descripción de la plataforma. La plataforma se describe en un archivo XML (o en código) con *hosts* y su velocidad, enlaces con ancho de banda y latencia, enrutadores, rutas y zonas de enrutamiento, que pueden anidarse para describir desde un clúster hasta una grilla de miles de nodos.

Validación. Casanova et al. (2014) muestran que SimGrid escala a millones de actores con costos de simulación moderados y discuten su precisión frente a ejecuciones reales y frente a otros simuladores. Para este trabajo, la conclusión práctica es que un simulador solo es tan bueno como su calibración: hay que medir el sistema real y ajustar los parámetros de la plataforma antes de creer en sus predicciones.

Vagrant

Vagrant es una herramienta de HashiCorp para crear y administrar entornos de máquinas virtuales a partir de un archivo declarativo, el `Vagrantfile`, escrito en un lenguaje de dominio específico basado en Ruby (Hashimoto, 2013; HashiCorp, s.f.). Sus conceptos principales son:

- **Box:** imagen base empaquetada de un sistema operativo (en este trabajo, `bento/ubuntu-24.04`).
- **Proveedor:** el hipervisor o plataforma que ejecuta las máquinas: VirtualBox, libvirt/KVM, Hyper-V, VMware o Docker.
- **Multimáquina:** un mismo `Vagrantfile` puede definir varias máquinas con nombre, recursos y red propios.
- **Redes:** puertos redirigidos, red privada (*host-only*) entre las máquinas y el anfitrión, o red pública en puente.
- **Carpetas sincronizadas y aprovisionadores:** el directorio del proyecto se comparte con las máquinas y, al crearlas, se ejecutan aprovisionadores (scripts de *shell*, Ansible, Puppet, Chef) que las configuran.

El ciclo de vida se maneja con pocos comandos: `vagrant up`, `vagrant ssh`, `vagrant halt`, `vagrant destroy` y `vagrant provision`.

Comparación con Proxmox y Docker. Proxmox VE es una plataforma de virtualización que se instala directamente sobre el hardware (KVM y contenedores LXC) y se

administra por web: sirve para un laboratorio permanente y compartido, pero requiere un servidor dedicado y su configuración no vive, por defecto, en un archivo versionado. Docker ejecuta contenedores que comparten el núcleo del anfitrión: arrancan en segundos y consumen mucha menos memoria, por lo que permitirían levantar decenas de "nodos" en una notebook (Merkel, 2014), pero todos comparten el mismo núcleo y el mismo planificador de CPU, lo que hace menos creíble la emulación de máquinas físicas distintas. Vagrant con máquinas virtuales completas queda en el medio: cada nodo tiene su núcleo, su pila de red y sus servicios `systemd`, y cada integrante puede reconstruir el laboratorio en su computadora. Para un laboratorio de cátedra, Proxmox conviene como infraestructura fija, Docker cuando lo que importa es la cantidad de nodos y Vagrant cuando importa reproducir máquinas completas en computadoras personales.

Cambio de licencia. En agosto de 2023 HashiCorp anunció que las versiones futuras de sus productos, Vagrant incluido, dejarían la licencia libre Mozilla Public License 2.0 y pasarían a la Business Source License 1.1 (HashiCorp, 2023). La BSL permite usar, modificar y redistribuir el código, salvo para ofrecer un producto que compita con los de HashiCorp, y cada versión pasa a una licencia libre cuatro años después de publicada. Para un uso académico o interno como el nuestro no cambia nada; las implicancias son de mediano plazo: Vagrant dejó de ser software libre en sentido estricto, su evolución depende de una sola empresa y la comunidad respondió con bifurcaciones de otros productos de la misma empresa (por ejemplo, OpenTofu como alternativa a Terraform). Por eso conviene que el laboratorio no dependa de funciones exclusivas de Vagrant: en nuestro caso toda la configuración está en Ansible y podría reutilizarse con otra herramienta que cree las máquinas.

Ansible

Ansible es una herramienta de gestión de configuración sin agentes: se conecta a los nodos por SSH y ejecuta módulos de Python que llevan cada recurso al estado descrito (Geerling, 2020; Red Hat, s.f.).

- El **inventario** lista los nodos, los agrupa (en nuestro caso, `coordinator` y `workers`) y les asigna variables.
- Un **playbook** es un archivo YAML con una o más jugadas; cada jugada aplica tareas a un grupo de nodos.
- Los **roles** empaquetan tareas, *handlers* (acciones que se disparan solo si algo cambió, como reiniciar un servicio), plantillas Jinja2 y archivos para reutilizarlos.
- La **idempotencia** es la propiedad central: las tareas describen un estado deseado (por ejemplo, "el paquete X instalado"), no una acción, y si el nodo ya está en ese estado no se modifica nada. Ejecutar un *playbook* idempotente dos veces seguidas debe terminar con cero cambios en la segunda.

Integración con Vagrant. Vagrant ofrece dos aprovisionadores: `ansible`, que ejecuta Ansible en el anfitrión, y `ansible_local`, que lo instala y ejecuta dentro de una máquina invitada. Como Windows no puede ser nodo de control de Ansible, usamos `ansible_local` en el coordinador, que aprovisiona a todos los nodos (incluido él mismo) por la red privada.

Middleware de comunicación

El modelo **productor-consumidor** desacopla a quien genera trabajo de quien lo procesa mediante una cola intermedia. En una cola de mensajes con *broker* como RabbitMQ, los productores publican en un *exchange*, el *broker* encola los mensajes y los entrega a los consumidores; con confirmaciones manuales (*acknowledgements*), un mensaje solo se borra cuando el consumidor confirma que lo procesó, y si el consumidor muere antes se le entrega

a otro. El parámetro *prefetch* limita cuántos mensajes sin confirmar puede tener cada consumidor.

ZeroMQ, en cambio, no tiene *broker*: es una biblioteca de *sockets* de mensajería con patrones predefinidos (REQ/REP, PUB/SUB, PUSH/PULL, DEALER/ROUTER) que transporta mensajes completos sobre TCP, con colas en memoria en cada extremo (Hintjens, 2013). No persiste mensajes ni implementa confirmaciones; si se necesitan, las programa la aplicación.

Elegimos ZeroMQ por tres razones: (a) en máquinas virtuales de 512 MB un *broker* de Erlang compite por memoria con la aplicación; (b) el robo de trabajo necesita comunicación directa entre trabajadores, que con un *broker* pasaría siempre por el coordinador; y (c) los mensajes explícitos entre *sockets* se corresponden uno a uno con los buzones de SimGrid, lo que facilita que el modelo simulado sea fiel al real. Las confirmaciones las implementamos en la aplicación: cada trabajador envía un mensaje `DONE` al terminar cada tarea y el coordinador no la da por terminada hasta recibirlo.

Emulación de red con tc y netem

En Linux, el subsistema de control de tráfico (`tc`) permite asociar disciplinas de encolado (*qdiscs*) a cada interfaz. La disciplina netem emula propiedades de una red de área amplia sobre una red local: retardo fijo, variación del retardo (*jitter*) con distintas distribuciones, pérdida, duplicación y reordenamiento de paquetes, y limitación de tasa (Hemminger, 2005). Por ejemplo, `tc qdisc replace dev eth1 root netem delay 20ms` retrasa 20 ms todos los paquetes que *salen* por `eth1`.

Hay dos detalles que importan para el diseño del experimento. Primero, netem actúa sobre el tráfico saliente: si se aplica el mismo retardo d en la interfaz de todos los nodos, cada mensaje sufre d una sola vez en su camino y la ida y vuelta suma $2d$. Segundo, netem retiene

los paquetes en una cola con límite (1000 por defecto), que con retardos altos puede llenarse y descartar ráfagas, por lo que conviene subirlo.

Monitoreo con Prometheus y Grafana

Prometheus es un sistema de monitoreo que recolecta métricas por *pull*: cada cierto intervalo consulta un punto HTTP de cada objetivo y guarda las series en una base de datos temporal, que se consulta con el lenguaje PromQL (Prometheus Authors, s.f.). Los *exporters* publican métricas de sistemas que no las exponen solos; `node_exporter` publica las de CPU, memoria, disco y red de un nodo Linux. Grafana se conecta a Prometheus y muestra tableros; tanto la fuente de datos como los tableros pueden aprovisionarse desde archivos, lo que permite versionarlos junto con el resto del laboratorio (Grafana Labs, s.f.).

Evaluación de desempeño

Jain (1991) propone un enfoque sistemático: fijar objetivos, definir el sistema y sus servicios, elegir métricas, listar parámetros, elegir cuáles serán **factores** (los que se varían a propósito) y con qué **niveles**, elegir la técnica, caracterizar la carga, diseñar los experimentos, analizarlos y presentar los resultados. Algunos conceptos que usamos:

- **Diseño factorial completo:** se prueban todas las combinaciones de niveles de los factores; permite estimar efectos e interacciones.
- **Repeticiones:** los sistemas reales tienen variabilidad, así que cada configuración se mide varias veces y se informa una media con su intervalo de confianza. Con pocas repeticiones el intervalo se calcula con la distribución *t* de Student.
- **Números aleatorios comunes:** usar las mismas semillas en todas las configuraciones hace que las diferencias entre estrategias no se confundan con diferencias en la carga sorteada.

- **Aleatorización del orden:** ejecutar las corridas en orden aleatorio evita que una deriva del entorno (por ejemplo, el anfitrión cada vez más cargado) se atribuya a un factor.

Jain (1991) también lista errores comunes: no tener objetivos claros, elegir métricas o cargas que no representan el problema, ignorar la variabilidad, no analizar la sensibilidad a los parámetros y presentar los resultados de forma engañosa. Los tuvimos en cuenta al definir el diseño de la sección de Materiales y métodos.

Organización del equipo

Diagnóstico de experiencia

Antes de comenzar, cada integrante completó la autoevaluación propuesta por la cátedra, con cinco dimensiones puntuadas de 0 a 3. La Tabla 3 muestra el resultado y el nivel que corresponde a cada total.

Tabla 3

Autoevaluación de experiencia de cada integrante (escala 0 a 3)

Dimensión	Quintana, Sofía	Salazar, Facundo	Vera, Romina
Linux y línea de comandos	1	3	2
Programación (Python, concurrencia, sockets)	1	3	3
Redes (TCP/IP, NAT, DNS, diagnóstico)	1	2	2
Virtualización y contenedores	1	3	3
Conceptos de SOD (Ejes 1 y 2)	1	2	2
Total (/15)	5	13	12
Nivel	Inicial	Avanzado	Avanzado

Nota. Escala de la cátedra: 0 a 5 puntos, nivel Inicial; 6 a 10, Intermedio; 11 a 15, Avanzado.

Roles y distribución de tareas

Los roles se asignaron según el nivel de cada integrante (Tabla 4). El acta de roles completa, con la conformidad de las tres personas, se incluye en el Anexo B.

Tabla 4

Roles asignados y entregables propios

Rol (nivel)	Integrante	Entregables propios
Investigador/a de modelado y simulación (Avanzado)	Salazar, Facundo	Pasos 1, 4 y 7: Vagrantfile multimáquina, modelo SimGrid y plataformas XML, extrapolación a gran escala. Además, video de demostración y presentación de la defensa
Ingeniero/a de infraestructura como código (Avanzado)	Vera, Romina	Pasos 2, 3 y 6: playbooks y roles de Ansible, emulación de red con netem, Prometheus y tablero de Grafana; módulo común de estrategias, coordinador y trabajadores; calibración y validación del simulador. Además, supervisión del paso 5 y compilación del informe
Analista de experimentación y datos (Inicial)	Quintana, Sofía	Pasos 0, 5 y 8: diagnóstico y acta de roles, planilla y ejecución del protocolo, prueba de reproducibilidad. Además, cuaderno Jupyter y README de reproducción

Además de sus entregables propios, el equipo acordó tres reglas de funcionamiento tomadas de la consigna: los pasos críticos se hacen en pares frente a la misma pantalla, alternando quién escribe (la primera puesta en marcha del clúster, a cargo de Salazar y Vera, y la calibración del simulador, a cargo de Salazar y Vera); cada integrante debe poder explicar la arquitectura completa en la defensa, no solo su parte; y todo cambio de código, configuración o datos pasa por el repositorio. Cada integrante llevó una bitácora semanal desde su rol (Anexo A).

Como Salazar y Vera quedaron en el nivel Avanzado, los roles entre ellos se asignaron por las dimensiones de mayor puntaje, según la regla del acta, y los dos hicieron juntos los pasos en pares. Los nueve pasos de la consigna se repartieron en partes iguales, tres por integrante: Vera tuvo a su cargo los pasos 2, 3 y 6 (Ansible, la implementación de las estrategias y la calibración y validación del simulador); Salazar, los pasos 1, 4 y 7 (Vagrant, modelo en SimGrid y extrapolación); y Quintana, los pasos 0, 5 y 8 (diagnóstico y acta, campaña real y reproducibilidad). Los entregables transversales también se repartieron: Vera supervisó la campaña del paso 5 y compiló el informe; Quintana armó el cuaderno de análisis

y redactó el README, como corresponde a su rol; y Salazar grabó el video y preparó la presentación de la defensa. El detalle está en docs/distribucion-de-tareas.md del repositorio.

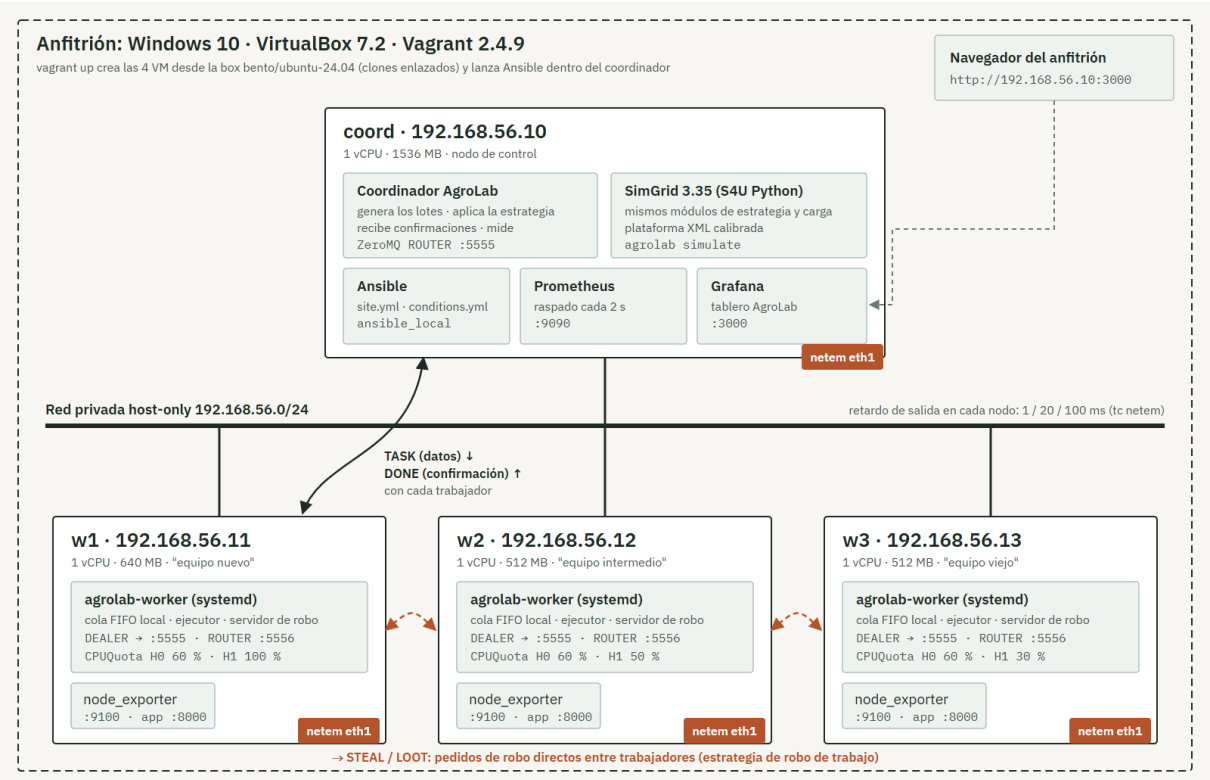
Materiales y métodos

Arquitectura del laboratorio

La Figura 1 muestra el laboratorio. Cuatro máquinas virtuales Ubuntu 24.04 comparten una red privada *host-only*: el coordinador (`coord`) y tres trabajadores (`w1`, `w2`, `w3`) con distinta memoria asignada, que representan un equipo nuevo, uno intermedio y uno viejo. Todo se crea con `vagrant up` y se configura con Ansible desde el coordinador.

Figura 1

Arquitectura del laboratorio AgroLab



Nota. Las cuatro máquinas virtuales se crean con `vagrant up` sobre VirtualBox. El retardo de netem se aplica a la salida de la interfaz privada de cada nodo; la velocidad de cada trabajador, con CPUQuota.

El coordinador cumple cuatro funciones: (a) ejecuta el **coordinador AgroLab**, que recibe los lotes de las estaciones, decide con la estrategia a qué trabajador va cada tarea y mide los resultados; (b) es el **nodo de control de Ansible**; (c) aloja **Prometheus y Grafana**;

y (d) ejecuta las **simulaciones de SimGrid**. Cada trabajador ejecuta el servicio `agrolab-worker`, que tiene una cola local de tareas, un hilo ejecutor y un servidor que atiende pedidos de robo de otros trabajadores.

Decisiones de diseño y alternativas consideradas

Un único módulo de decisión. La lógica de cada estrategia (`strategies.py`) y la generación de la carga (`workload.py`) son módulos de Python que importan tanto el coordinador real como el simulador. Así, la consigna de que "ambos mundos ejecuten exactamente el mismo algoritmo" se cumple por construcción: no hay dos implementaciones que puedan divergir.

ZeroMQ en lugar de RabbitMQ. Lo justificamos en la sección de Middleware de comunicación: menor consumo de memoria, comunicación directa entre trabajadores para el robo y correspondencia directa con los buzones de SimGrid.

Heterogeneidad con CPUQuota. La velocidad de cada trabajador se controla con el límite de CPU de *cgroups* v2 que ofrece `systemd` (`CPUQuota`), aplicado al servicio del trabajador. Consideramos dos alternativas: asignar distinta cantidad de vCPU (no cambia nada porque cada trabajador ejecuta una tarea por vez) y el límite de ejecución de VirtualBox (`cpuexecutioncap`), que funciona solo con ese proveedor. CPUQuota se cambia en caliente, sin reiniciar el servicio, y funcionaría igual con libvirt. Para que la ejecución limitada sea pareja bajamos el período de *cgroups* de 100 ms a 10 ms.

Dos niveles de heterogeneidad con la misma capacidad total. En el nivel homogéneo (H0) los tres trabajadores corren al 60 % de un núcleo; en el heterogéneo (H1), al 100 %, 50 % y 30 %. Ambos suman 180 % de un núcleo, de modo que la comparación aísla el efecto de *cómo* está repartida la capacidad, sin confundirlo con *cuánta* hay.

netem en todos los nodos. El retardo se aplica a la salida de la interfaz privada de los cuatro nodos. Así, cualquier mensaje (coordinador a trabajador, trabajador a coordinador o entre trabajadores) sufre el retardo una sola vez y la ida y vuelta suma el doble.

`ansible_local` en lugar de Ansible en el anfitrión. Windows no puede ser nodo de control de Ansible; ejecutarlo dentro del coordinador hace que el laboratorio funcione igual en cualquier anfitrión.

Una sola vCPU por máquina. En el anfitrión usado, Windows con Hyper-V activo, VirtualBox se ejecuta sobre la interfaz de virtualización de Hyper-V, y las máquinas con dos vCPU quedaban colgadas al azar durante el arranque. Con una vCPU arrancan siempre (ver Implementación práctica, paso 1). Como todas las aplicaciones del laboratorio son de un solo hilo, la restricción no afecta al experimento.

Software y hardware

Tabla 5

Software y hardware del laboratorio

Componente	Versión o detalle
Anfitrión	AMD Ryzen 5 3600 (6 núcleos, 12 hilos), 16 GB de RAM, Windows 10 Pro con Hyper-V activo
Hipervisor y Vagrant	VirtualBox 7.2.20; Vagrant 2.4.9
Sistema invitado	Ubuntu 24.04.3 LTS (box bento/ubuntu-24.04 v202510.26.0), 1 vCPU por VM
Memoria por VM	coord 1536 MB; w1 640 MB; w2 y w3 512 MB
Aprovisionamiento	ansible-core 2.16 con el aprovisionador ansible_local
Aplicación	Python 3.12, pyzmq 24 (libzmq 4.3), prometheus-client 0.19
Simulación	SimGrid 3.35 (S4U, Python), modelo de red CM02
Monitoreo	Prometheus y prometheus-node-exporter (paquetes de Ubuntu); Grafana OSS (repositorio de Grafana Labs)
Análisis	Jupyter, pandas, matplotlib y SciPy en el anfitrión (Python 3.11)

Carga de trabajo

Cada corrida simula 12 períodos de llegada separados por 3 s; en cada período llega un lote de 8 tareas al coordinador (96 tareas por corrida). Cada tarea es, con probabilidad 0,75, una **lectura de sensores** (liviana, pocos datos) y, con probabilidad 0,25, una **imagen de**

cultivo (pesada, más datos). El costo de CPU y el tamaño de los datos de cada tarea se sortean uniformemente en ±50 % alrededor de la media de su tipo (Tabla 6), es decir, la distribución del costo es bimodal. La unidad de trabajo es el cálculo de SHA-256 sobre un bloque de 32 KiB: es determinista, usa un solo núcleo y su duración depende solo de la velocidad del procesador.

Tabla 6

Parámetros de la carga de trabajo

Parámetro	Lectura de sensores	Imagen de cultivo
Probabilidad	0,75	0,25
Costo medio (unidades de trabajo)	11.000	88.000
Datos medios por tarea	16 KiB	256 KiB
Dispersión de costo y datos	±50 % (uniforme)	±50 % (uniforme)
Lotes por corrida / tareas por lote	12 / 8	
Período entre lotes	3 s	

Nota. Una unidad de trabajo es el cálculo de SHA-256 sobre un bloque de 32 KiB.

La carga se genera con `random.Random(semilla)`, de modo que una misma semilla produce la misma secuencia de tareas en el clúster y en el simulador. Con los parámetros elegidos, la demanda media es de alrededor del 82 % de la capacidad total del clúster en cualquiera de los dos niveles de heterogeneidad; es una carga alta, que es donde las estrategias se diferencian.

Estrategias implementadas

Las tres estrategias se implementan en el módulo común `strategies.py`:

- `RoundRobin.place()` devuelve el siguiente trabajador en orden cíclico.
- `LeastLoaded.place()` recorre los *N* contadores de tareas pendientes que mantiene el coordinador (se incrementan al despachar y se decrementan al recibir la confirmación) y devuelve el mínimo; los empates se resuelven en forma rotativa.
- `WorkStealing` reparte como Round Robin y le agrega a cada trabajador una `StealPolicy`: cuando su cola queda vacía (política de transferencia), elige una

víctima uniforme al azar entre los demás (ubicación) y le pide la mitad, redondeando hacia arriba, de las tareas que todavía no empezaron, tomadas del final de su cola (selección). Si la víctima no tiene nada, espera un tiempo que se duplica en cada intento fallido, de 50 a 400 ms, antes de volver a intentar.

Diseño experimental

Tabla 7

Diseño experimental

Elemento	Definición
Factor principal: estrategia	(1) Round Robin estático; (2) Menos cargado, centralizado; (3) Robo de trabajo
Factor: latencia de red	1, 20 y 100 ms de retardo de salida con netem en todos los nodos
Factor: heterogeneidad de CPU	H0: 60/60/60 %; H1: 100/50/30 % de un núcleo (misma capacidad total)
Factor: cantidad de nodos	3 trabajadores en el clúster real; 3 a 512 en simulación
Carga	Bimodal, 12 lotes de 8 tareas, semilla fija por repetición
Métricas	Makespan, rendimiento, espera media, CV de ocupación, mensajes de control
Repeticiones	5 por configuración (90 corridas reales); media, desvío e IC 95 %
Validación	Error relativo |simulado − real| / real por configuración

El diseño es factorial completo: 3 estrategias × 3 latencias × 2 niveles de heterogeneidad = 18 configuraciones, con 5 repeticiones cada una (90 corridas en el clúster real). La repetición *r* usa la semilla 999 + *r* en todas las configuraciones (números aleatorios comunes). Las corridas se agrupan en 6 bloques, uno por combinación de latencia y heterogeneidad, para no reconfigurar la red y la CPU en cada corrida; dentro de cada bloque, el orden de las 15 corridas se sorteó con una semilla fija. La planilla de corridas resultante (`experiments/planilla.csv`) es la que siguió el rol de experimentación.

Métricas

Todas las métricas se calculan con el reloj del coordinador (o el reloj simulado), relativas al inicio de la corrida, para no comparar relojes de máquinas distintas. Para la tarea

i, sea a_i su llegada al coordinador, c_i la recepción de su confirmación y s_i su tiempo de ejecución medido en el trabajador:

- **Makespan:** max c_i, el tiempo hasta que se confirma la última tarea.
- **Rendimiento:** tareas completadas por segundo de makespan.
- **Tiempo medio de espera:** media de $(c_i - a_i - s_i)$, el tiempo que la tarea pasó sin ejecutarse (en colas o viajando por la red).
- **Desbalance de carga:** coeficiente de variación del tiempo de ocupación de cada trabajador (suma de los s_i que ejecutó). Si todos los trabajadores estuvieron ocupados el mismo tiempo el CV es 0, aunque sean de distinta velocidad.
- **Mensajes de control:** confirmaciones más pedidos y respuestas de robo; los mensajes que llevan las tareas se cuentan aparte.

Validación

Para cada configuración comparable se calcula el error relativo |simulado − real| / real, tomando como valor real la media de las 5 repeticiones y como simulado la media de las 5 simulaciones con las mismas semillas.

Modelo de simulación

El modelo en SimGrid (`sim.py`) reproduce la estructura del sistema real con actores de S4U en Python:

- En el *host* `coord`, un actor **despacho** espera la llegada de cada lote (`sleep_until`), consume el costo de CPU de decidir y enviar cada tarea, llama a la misma estrategia y envía la tarea al buzón del trabajador con un tamaño igual a encabezado más datos. Un actor **recepción** recibe las confirmaciones y consume el costo de procesarlas. Ambos comparten un único núcleo, como el hilo único del coordinador real.

- En cada trabajador, un actor **buzón** recibe tareas y las encola; un actor **ejecutor** las ejecuta en orden (`execute` con el costo de la tarea) y envía la confirmación; y un actor **servidor de robo** atiende los pedidos de otros trabajadores con la misma política de selección.

La plataforma es una estrella: cada *host* tiene un enlace propio hasta un conmutador, como la red *host-only*. Un mensaje entre dos *hosts* atraviesa dos enlaces, así que cada enlace lleva la mitad de la latencia de ida medida. Usamos el modelo de red CM02, en el que un mensaje solo tarda latencia más tamaño dividido el ancho de banda disponible, porque los factores de corrección del modelo por defecto (LV08) están calibrados para transferencias TCP de gran volumen y nuestros mensajes van de 160 bytes a 400 KiB (Velho et al., 2013).

La calibración de la red merece una aclaración. Al aumentar el retardo de netem, el ancho de banda efectivo medido cae de 34 MB/s a menos de 0,5 MB/s; eso no significa que la red sea más lenta, sino que cada conexión TCP no puede tener en vuelo más que su ventana de congestión, de modo que rinde como máximo ventana / RTT, y las conexiones que estuvieron inactivas vuelven a arrancar con una ventana chica. Si usáramos ese valor como capacidad del enlace, el simulador concluiría que a gran escala el enlace del coordinador se satura con muy poco tráfico. Por eso el enlace del coordinador, que comparten todas las conexiones, tiene la capacidad medida sin netem, y el enlace de cada trabajador tiene el rendimiento de *una* conexión en ese nivel de latencia: los mensajes hacia un mismo trabajador viajan por la misma conexión de ZeroMQ y se reparten ese límite. La ventana efectiva que resulta (rendimiento × 2 × latencia) es casi la misma en los cuatro niveles medidos, entre 101 y 141 KiB, lo que confirma que el límite es de la conexión y no de la red.

Los demás parámetros de la plataforma (velocidad de cada *host* y costo del coordinador por mensaje) salen de la calibración del paso 6.

Implementación práctica

Todo el laboratorio está en el repositorio https://github.com/RominaaVeraa/tif-sod-vagrant-simgrid. Esta sección recorre los pasos de la consigna con las evidencias de cada uno; los fragmentos de configuración son extractos de los archivos del repositorio.

Paso 0: diagnóstico y acta de roles

Completamos la autoevaluación, asignamos los roles (Tablas 3 y 4), creamos el repositorio y verificamos qué computadoras podían ejecutar Vagrant. La computadora usada para el desarrollo y las mediciones tiene un procesador AMD Ryzen 5 3600 (6 núcleos, 12 hilos), 16 GB de RAM y Windows 10 con Hyper-V activo, porque se usan WSL2 y Docker Desktop; esto último tuvo consecuencias en el paso 1.

Paso 1: clúster como código con Vagrant

El `Vagrantfile` define los trabajadores a partir de una variable `N_WORKERS` (3 por defecto) y genera, con la misma definición, el inventario de Ansible. Los trabajadores se declaran antes que el coordinador para que Vagrant los cree primero: cuando llega al coordinador ya existen las claves SSH que Ansible necesita para conectarse a ellos.

```
N_WORKERS = Integer(ENV.fetch("N_WORKERS", "3"))
WORKER_PROFILES = [
  { memory: 640 },  # equipo nuevo
  { memory: 512 },  # equipo intermedio
  { memory: 512 },  # equipo viejo
]
workers = (1..N_WORKERS).map do |i|
  { name: "w#{i}", ip: "#{NET}.#{10 + i}", index: i - 1 }
    .merge(WORKER_PROFILES[(i - 1) % WORKER_PROFILES.size])
end
...
config.vm.define "coord", primary: true do |node|
  node.vm.network "private_network", ip: COORD_IP
  node.vm.synced_folder ".", "/vagrant",
                        mount_options: ["dmode=755", "fmode=644"]
  node.vm.provision "bootstrap", type: "shell",
                    path: "provisioning/bootstrap-coord.sh"
  node.vm.provision "ansible", type: "ansible_local" do |ansible|
    ansible.playbook = "site.yml"
    ansible.limit    = "all"
  end
end
```

La primera puesta en marcha mostró tres problemas que no estaban en ninguna guía y que documentamos en el `README.md`:

1. **Una máquina virtual huérfana bloqueaba Vagrant.** VirtualBox tenía registrada una VM de otra materia cuyos archivos ya no existían. Antes de redirigir puertos, Vagrant consulta los puertos usados por *todas* las VM registradas y esa consulta fallaba con `E_ACCESSDENIED`. Se resolvió desregistrándola (`VBoxManage unregistervm`).
2. **Las máquinas con dos vCPU se colgaban al arrancar.** Con Hyper-V activo, VirtualBox no usa directamente las extensiones de virtualización del procesador sino la interfaz de Hyper-V. En esas condiciones, las VM con dos vCPU quedaban detenidas en "Loading essential drivers" con un vCPU al 100 %, en forma intermitente. Probamos por separado el chipset (ICH9 y PIIX3), la paravirtualización (KVM y Hyper-V) y la cantidad de vCPU: solo con una vCPU arrancaron siempre (Tabla 8). Como todas las aplicaciones del laboratorio son de un solo hilo, fijamos una vCPU por máquina.
3. **Permisos de la carpeta compartida.** En un anfitrión Windows, la carpeta del proyecto aparece dentro de la VM con permisos 777. Ansible se niega a leer un `ansible.cfg` en un directorio escribible por todos e intenta *ejecutar* el inventario como si fuera un script. Montamos la carpeta con `dmode=755,fmode=644` y copiamos las claves privadas de los trabajadores a `~/.ssh` con permisos 600.

Tabla 8

Pruebas de arranque de una VM con Hyper-V activo en el anfitrión

vCPU	Chipset	Paravirtualización	Arranques completos / intentos
2	ICH9 (el de la box)	KVM	0 / 1
2	PIIX3	KVM	1 / 2
2	PIIX3	Hyper-V	2 / 3
1	PIIX3	KVM	1 / 1
1	ICH9 (configuración final)	KVM	16 / 16

Nota. Un intento fallido es una VM detenida en «Loading essential drivers» con un vCPU al 100 % durante más de 4 minutos.

Con esos cambios verificamos el ciclo completo sin intervención manual: `vagrant up`, `vagrant ssh`, `vagrant halt`, `vagrant up` y `vagrant destroy -f` seguido de un nuevo `vagrant up`. En el clúster usado para la campaña, `vagrant halt` tardó 1,5 minutos y el nuevo `vagrant up` de las cuatro máquinas ya creadas, 4,5 minutos; `vagrant destroy -f` las eliminó en 15 segundos. En el clúster reconstruido del paso 8 repetimos el apagado y el encendido y, sin tocar nada, la misma corrida de prueba volvió a funcionar: los servicios de los trabajadores arrancan solos porque están habilitados en `systemd`. Desde que fijamos una vCPU por máquina, las VM arrancaron en los 16 intentos (Tabla 8).

Paso 2: aprovisionamiento con Ansible

El *playbook* `site.yml` aplica cuatro roles: `common` (paquetes, `node_exporter`, nombres de los nodos y scripts de condiciones) en todos los nodos; `worker` (usuario de servicio, copia del código y unidad `systemd`) en los trabajadores; y `coordinator` (SimGrid y el comando `agrolab`) y `monitoring` (Prometheus y Grafana con su fuente de datos y tablero) en el coordinador. Todas las tareas usan módulos declarativos; las dos que ejecutan comandos (aplicar netem y CPUQuota) informan si cambiaron algo, para no romper la idempotencia.

```
- name: Unidad systemd del trabajador
  ansible.builtin.template:
    src: agrolab-worker.service.j2
    dest: /etc/systemd/system/agrolab-worker.service
  notify: Reiniciar trabajador

# agrolab-worker.service.j2
ExecStart=/usr/bin/python3 -m agrolab.worker           --name {{ inventory_hostname
}} --coord {{ coord_ip }} ...
CPUQuota={{ cpu_quota_default }}%
CPUQuotaPeriodSec={{ cpu_quota_period }}
```

La segunda ejecución del *playbook* terminó sin cambios en ningún nodo:

```
PLAY RECAP
coord                      : ok=20   changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
w1                         : ok=12   changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
w2                         : ok=12   changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
w3                         : ok=12   changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

El *playbook* `conditions.yml` aplica las condiciones de cada bloque de corridas: el retardo de netem en los cuatro nodos y la CPUQuota de cada trabajador, que se cambia en caliente con `systemctl set-property --runtime` sin reiniciar el servicio.

Paso 3: implementación de las estrategias

El paquete `agrolab` tiene diez módulos; los dos centrales son comunes a ambos mundos:

- `workload.py` genera la carga a partir de una semilla.
- `strategies.py` contiene las tres estrategias y la política del ladrón.

El **trabajador** real tiene dos hilos. El de comunicaciones recibe tareas del coordinador por un *socket* DEALER, atiende pedidos de robo por un ROUTER propio y reenvía al coordinador las confirmaciones que le pasa el ejecutor. El **ejecutor** saca tareas de la cola local y las ejecuta; si la estrategia es robo de trabajo y la cola está vacía, envía un pedido `STEAL` a una víctima y espera la respuesta `LOOT` con las tareas robadas y sus datos. El núcleo de cómputo (SHA-256) libera el bloqueo global del intérprete de Python mientras calcula, así que el hilo de comunicaciones sigue atendiendo mensajes durante la ejecución.

El **coordinador** real es un único hilo con un bucle de eventos: despacha cada lote en su instante de llegada, consulta la estrategia para cada tarea y, entre lotes, procesa las confirmaciones. Antes de cada corrida verifica que los trabajadores ejecuten la misma versión del código comparando un *hash* del paquete.

```
def place(self, task, view):          # LeastLoaded, en strategies.py
    best, best_load = -1, None
    for k in range(self.n):
        w = (self._start + k) % self.n
        load = view.outstanding[w]
        if best_load is None or load < best_load:
            best, best_load = w, load
    self._start = (best + 1) % self.n
    return best
```

Antes de usar las máquinas virtuales probamos el protocolo con coordinador y tres trabajadores en una misma computadora (`tests/smoke_local.py`). Esa prueba encontró un error real: la función que arma los mensajes tenía un parámetro llamado `kind`, el mismo nombre que un campo del mensaje de tarea.

Paso 4: modelo en SimGrid

El modelo describe una plataforma equivalente al clúster (Anexo D) y programa con S4U los mismos actores que el sistema real (sección Modelo de simulación). La plataforma se genera a partir de la calibración: para cada nivel de heterogeneidad se toman las velocidades medidas de cada trabajador y, para cada nivel de netem, la latencia y el ancho de banda ajustados. Para escalas mayores que la del laboratorio se repite el patrón de velocidades de los tres trabajadores.

La verificación con un caso trivial (una tarea de 25 000 unidades con 256 KiB de datos, un trabajador de 50 000 u/s y la red de 100 ms) dio el mismo tiempo que la cuenta hecha a mano:

```
Plataforma: L = 155.235 ms, B coordinador = 34.1 MB/s, B trabajador = 0.466 MB/s
(ventana 141.2 KiB), trabajador a 50000 u/s
  despacho en el coordinador                                    0.219 ms
  envío de la tarea: L + (encabezado + datos) / min(B, Bw)    718.559 ms
  ejecución: costo / velocidad                                500.000 ms
  confirmación: L + encabezado / min(B, Bw)                   155.579 ms
  total calculado a mano                                     1374.357 ms
  total simulado (makespan)                                  1374.357 ms
  diferencia relativa: 1.19e-07
```

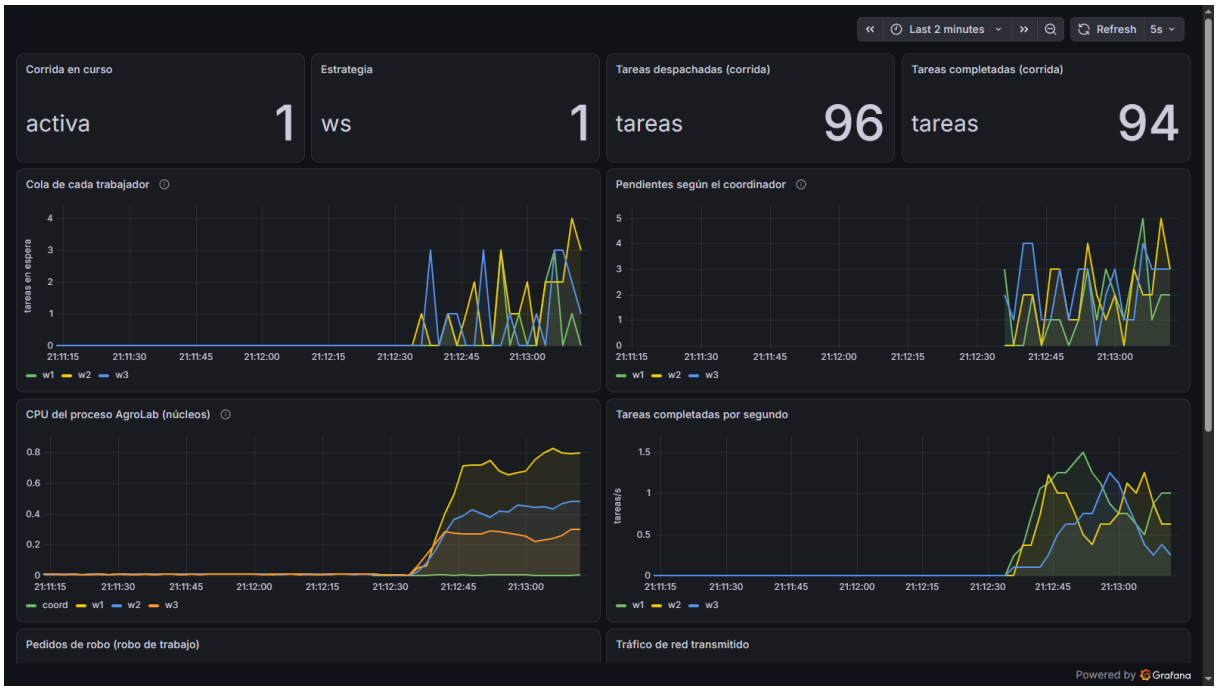
Durante la programación del modelo aparecieron dos diferencias entre la documentación general de SimGrid y la versión 3.35 empaquetada en Ubuntu: una comunicación asíncrona ya iniciada no puede "desprenderse" (`detach`), por lo que los envíos sin espera se crean con `put_init(...).detach()`; y como SimGrid admite un único motor de simulación por proceso, cada simulación corre en un proceso nuevo. Para que los mensajes que llegan a un mismo buzón no se serialicen artificialmente (en SimGrid una transferencia normalmente empieza recién cuando el receptor la pide), cada actor receptor se declara receptor permanente de su buzón (`set_receiver`), igual que un *socket* TCP que recibe datos aunque la aplicación todavía no los haya leído.

Paso 5: experimentación en el clúster real

El comando `agrolab run` ejecuta la planilla: antes de cada bloque aplica las condiciones con `conditions.yml`, mide la ida y vuelta real con mensajes vacíos y luego ejecuta las 15 corridas del bloque en el orden sorteado, agregando una fila por corrida a `runs.csv` y una por tarea a `tasks.csv`. Si se interrumpe, al volver a ejecutarlo saltea las corridas que ya están en el archivo. La campaña completa de 90 corridas llevó 66 minutos. Durante la campaña el anfitrión siguió en uso normal; las condiciones de cada bloque se verificaron midiendo la ida y vuelta de mensajes vacíos antes de empezarlo, y quedaron registradas en el CSV. La Figura 2 muestra el tablero de Grafana durante una de esas corridas.

Figura 2

Tablero de Grafana durante una corrida de robo de trabajo (H1, 20 ms)



Nota. Captura del clúster reconstruido desde cero (paso 8). Arriba, el estado de la corrida que publica el coordinador; en el medio, la cola de cada trabajador y las tareas pendientes según el coordinador; abajo, la CPU del proceso de cada nodo, que sigue las cuotas de 100, 50 y 30 %.

Paso 6: calibración y validación

La calibración (`agrolab calibrate`) mide la velocidad de cada trabajador con una tarea de referencia de 20 000 unidades en cada nivel de heterogeneidad (Tabla 9), el tiempo de ida de mensajes de 0 a 256 KiB en cada nivel de netem (Tabla 10 y Figura 3) y el costo de CPU del coordinador por mensaje en una corrida que no forma parte del diseño (219 µs por mensaje). Además, un microbenchmark de `LeastLoaded.place()` midió 82,8 ns por trabajador examinado, que el simulador usa para escalar el costo de esa decisión con *N*.

Tabla 9

Velocidad medida de cada trabajador con la tarea de referencia (unidades de trabajo por segundo)

Nivel	w1	w2	w3	Total
Sin límite (100 %)	54.177	52.584	53.980	160.741
H0 (60/60/60 %)	32.638	33.507	33.240	99.385
H1 (100/50/30 %)	54.153	27.835	16.220	98.208

Nota. Mediana de 5 ejecuciones de 20.000 unidades. El coordinador mide 52.477 u/s.

Tabla 10

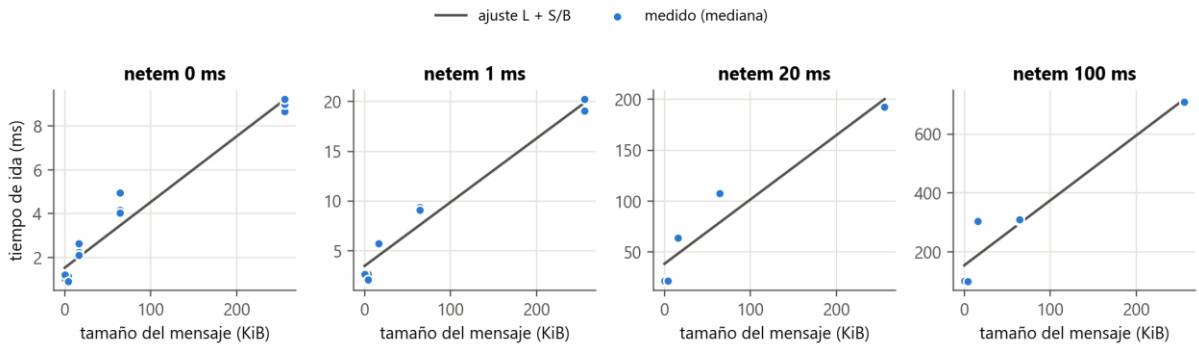
Parámetros de red ajustados a partir de las mediciones de ida y vuelta

netem	Latencia de ida L (ms)	Ancho de banda B (MB/s)	B (Mbit/s)	R^2
0 ms	1,52	34,1	273	0,958
1 ms	3,47	16,0	128	0,960
20 ms	38,36	1,6	13	0,914
100 ms	155,24	0,5	4	0,919

Nota. Ajuste por mínimos cuadrados de t(S) = L + S / B con mensajes de 0 a 256 KiB enviados después de 0,6 s de inactividad.

Figura 3

Tiempo de ida de un mensaje según su tamaño, medido y ajustado



Las velocidades confirman que CPUQuota hace lo que promete: sin límite, las tres VM rinden entre 52 600 y 54 200 unidades por segundo, aunque tienen distinta memoria; con el 60 % rinden entre el 60 y el 64 % de su velocidad libre, y con 100/50/30 %, el 100, 53 y 30 %. La red mostró un comportamiento que obligó a revisar el modelo. Un mensaje vacío tarda en ir 1,1 ms sin netem y 2,6, 21,8 y 102 ms con 1, 20 y 100 ms de retardo, lo esperable. Pero

el tiempo de los mensajes con datos crece en escalones: con 100 ms, uno de 16 KiB tarda dos idas y vueltas y uno de 256 KiB, cuatro. Es el arranque lento de TCP, que vuelve a empezar con una ventana chica en cada conexión que estuvo inactiva, como ocurre entre lote y lote. El ajuste lineal (Figura 3) resume ese comportamiento en una latencia y un rendimiento por conexión para cada nivel, con un R^2 de 0,91 a 0,96, y, como se explicó en Materiales y métodos, ese rendimiento se modela como límite de cada conexión y no como capacidad de la red. El costo del coordinador fue de 219 µs de CPU por mensaje sin la instrumentación de Prometheus; durante la campaña, con las métricas activas, fue de unos 380 µs.

Paso 7: extrapolación a gran escala

Con la plataforma calibrada simulamos 3, 16, 64, 256 y 512 trabajadores, con las tres estrategias, los tres niveles de latencia, los dos de heterogeneidad y 3 semillas (270 simulaciones, 37 minutos), y después 18 simulaciones más con 384, 640 y 768 trabajadores y 20 ms de latencia para ubicar con más precisión el límite del coordinador. Para que la comparación entre escalas sea justa, la carga crece en proporción a la cantidad de trabajadores: con N trabajadores cada lote trae $8N/3$ tareas, de modo que la utilización nominal es la misma que en el clúster real. Los resultados se discuten en la sección siguiente.

Paso 8: prueba de reproducibilidad

Antes de la prueba en otra computadora, que corresponde al rol de experimentación y se registra en la planilla del repositorio, hicimos una prueba en la computadora de desarrollo: destruimos el clúster, clonamos el repositorio desde GitHub en una carpeta nueva y seguimos solamente el `README.md`. El `vagrant up` terminó en 18,4 minutos sin errores ni intervención manual; la segunda ejecución de Ansible dio `changed=0` en los cuatro nodos; y la corrida de prueba (robo de trabajo, H1, 20 ms, semilla 1000) dio un makespan de 40,41 s, contra 40,50 s de la misma corrida en la campaña original, una diferencia del 0,2 %. La simulación equivalente dio 40,03 s. Esta prueba no reemplaza a la de otra computadora, porque la de

desarrollo ya tenía descargada la imagen base y resueltos los problemas de su entorno; esa prueba, a cargo de Quintana, se describe a continuación.

La prueba en otra computadora la hicieron Quintana y Vera el 26 de octubre, en una notebook de Vera que no participó del desarrollo (AMD Ryzen 7 2700U de 4 núcleos, 16 GB de RAM, Windows 11 sin Hyper-V, VirtualBox 7.1.6 y Vagrant 2.4.9), siguiendo solamente el README y registrando cada paso en docs/prueba-reproducibilidad.md. El vagrant up desde un clon nuevo, con la descarga de la imagen base incluida, terminó en 11,1 minutos sin intervención; la segunda ejecución de Ansible dio changed=0 en los cuatro nodos; Grafana y Prometheus respondieron; y el ciclo de apagado y encendido funcionó (0,6 y 3,2 minutos). Con la misma semilla, las corridas reales dieron 49,7 s con robo de trabajo, 52,0 s con Menos cargado y 73,6 s con Round Robin, contra 40,5, 43,9 y 60,9 s de la campaña original: el mismo orden de estrategias, pero unos 20 % más lentas, porque la CPU de la notebook es más lenta que la de la computadora de desarrollo, sobre la que está calibrado el simulador (que dio 40,0, 41,9 y 61,5 s). Una corrida repetida justo después de reiniciar el clúster dio 66,7 s, lo que indica que conviene esperar a que las máquinas terminen de arrancar antes de medir.

La prueba encontró dos problemas que no habían aparecido en la computadora de desarrollo. Un primer intento, hecho con la notebook a batería, dio tiempos casi el doble de largos porque Windows reduce la frecuencia de la CPU, y se descartó. Además, vagrant destroy deja en la carpeta de VirtualBox un directorio vacío por máquina, con certificados que crea VirtualBox 7.1, y eso hace fallar el siguiente vagrant up con VERR_ALREADY_EXISTS. Los dos quedaron documentados en los problemas conocidos del README. La prueba confirma lo que discutimos en la validación: el laboratorio se reproduce en otra computadora y el orden de las estrategias se mantiene, pero los valores

absolutos dependen del anfitrión, y una calibración solo vale para la computadora en la que se hizo.

Análisis de resultados y discusión

Desempeño y comparación de las estrategias

La campaña real completó las 90 corridas del diseño en 66 minutos, sin fallas de comunicación ni corridas trabadas. Tres corridas del primer bloque se repitieron al final porque, mientras se ejecutaban, cambiaron archivos del simulador en el repositorio y la columna que registra la versión del código quedó distinta, aunque el código medido era el mismo; las filas originales se conservan aparte. Las repeticiones dieron 36,41, 37,18 y 38,20 s de makespan contra 36,36, 37,18 y 38,27 s de las originales, una primera muestra de que, con la misma semilla, el laboratorio es muy repetible. La Tabla 11 resume las métricas por configuración.

Tabla 11

Resultados en el clúster real por configuración (media de 5 repeticiones)

Het.	Lat. (ms)	Estrategia	Makespan (s)	Rend. (t/s)	Espera (s)	CV ocup.	Msj. control
H0	1	Menos cargado	39,38 ± 2,95	2,45	1,43 ± 0,27	0,156	96
H0	1	Round Robin	39,15 ± 4,11	2,46	1,60 ± 0,22	0,131	96
H0	1	Robo de trabajo	37,43 ± 1,61	2,57	0,90 ± 0,19	0,058	394
H0	20	Menos cargado	38,94 ± 2,49	2,47	1,49 ± 0,19	0,107	96
H0	20	Round Robin	39,21 ± 3,92	2,46	1,68 ± 0,22	0,134	96
H0	20	Robo de trabajo	37,34 ± 1,36	2,57	1,14 ± 0,30	0,073	363
H0	100	Menos cargado	38,85 ± 1,62	2,47	2,05 ± 0,11	0,088	96
H0	100	Round Robin	39,87 ± 4,04	2,42	2,22 ± 0,22	0,134	96
H0	100	Robo de trabajo	38,67 ± 2,51	2,49	1,98 ± 0,36	0,044	255
H1	1	Menos cargado	42,64 ± 6,16	2,28	2,36 ± 0,66	0,189	96
H1	1	Round Robin	56,44 ± 8,39	1,72	4,87 ± 1,99	0,407	96
H1	1	Robo de trabajo	37,90 ± 1,91	2,54	1,26 ± 0,35	0,065	348
H1	20	Menos cargado	42,03 ± 7,44	2,32	2,35 ± 0,64	0,157	96
H1	20	Round Robin	56,69 ± 8,41	1,71	5,02 ± 1,98	0,408	96
H1	20	Robo de trabajo	37,72 ± 2,16	2,55	1,46 ± 0,48	0,081	301
H1	100	Menos cargado	43,05 ± 7,87	2,27	3,07 ± 0,55	0,178	96
H1	100	Round Robin	57,48 ± 8,82	1,69	5,58 ± 2,06	0,412	96

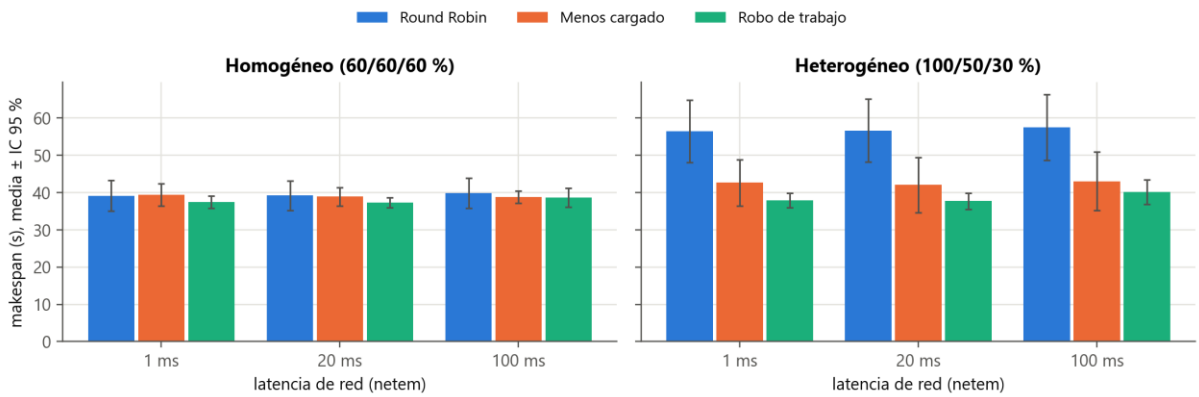
Het.	Lat. (ms)	Estrategia	Makespan (s)	Rend. (t/s)	Espera (s)	CV ocup.	Msj. control
H1	100	Robo de trabajo	40,10 ± 3,29	2,40	2,53 ± 0,44	0,100	222

Nota. Los valores con ± indican el semiancho del intervalo de confianza del 95 % (t de Student, 4 grados de libertad); el desvío estándar de cada métrica está en analysis/tablas/real_resumen.csv. Rend. = rendimiento en tareas por segundo; CV ocup. = coeficiente de variación del tiempo de ocupación entre trabajadores.

Makespan. La Figura 4 muestra el resultado central. Con los tres trabajadores a la misma velocidad (H0), las tres estrategias terminan en 37 a 40 s y sus diferencias no son estadísticamente significativas (Tabla 12: $p > 0{,}39$ en las tres latencias): el makespan queda dominado por el último lote, que llega a los 33 s, y cualquier reparto razonable lo termina en pocos segundos. Con la misma capacidad total repartida en forma desigual (H1), en cambio, las diferencias son grandes y significativas ($p < 0{,}001$): Round Robin tarda entre 56 y 57 s, Menos cargado entre 42 y 43 s y Robo de trabajo entre 38 y 40 s, prácticamente lo mismo que en el caso homogéneo.

Figura 4

Makespan en el clúster real según estrategia, latencia y heterogeneidad



Nota. Barras: media de 5 repeticiones; líneas: IC 95 %.

Tabla 12

ANOVA de un factor (estrategia) sobre el makespan, por configuración

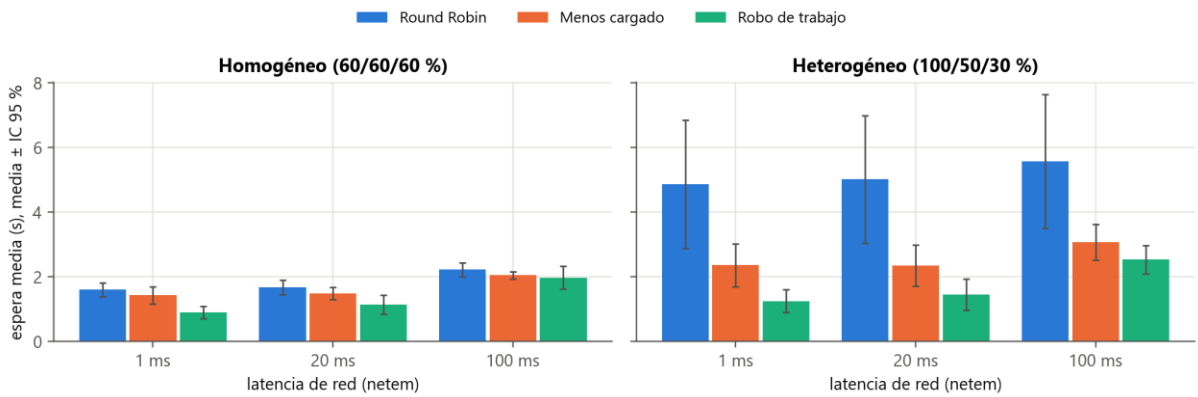
Heterogeneidad	Latencia (ms)	F	p	Mejor estrategia
H0	1	0,93	0,421	Robo de trabajo
H0	20	1,01	0,393	Robo de trabajo
H0	100	0,38	0,690	Robo de trabajo
H1	1	19,15	< 0,001	Robo de trabajo
H1	20	17,48	< 0,001	Robo de trabajo
H1	100	13,28	< 0,001	Robo de trabajo

Nota. Tres grupos de 5 corridas por configuración (2 y 12 grados de libertad).

La razón se ve en cómo se repartió el trabajo. Round Robin le dio 32 tareas a cada trabajador, sin importar su velocidad: en H1 el trabajador al 30 % acumuló en promedio 54,6 s de ejecución contra 20,1 s del que corre al 100 %, y el makespan es el de ese trabajador lento. Menos cargado y Robo de trabajo le asignaron al trabajador rápido unas 42 y 44 tareas y al lento unas 20, en una proporción cercana a la de sus velocidades (54 : 28 : 16). Menos cargado no alcanza al robo de trabajo porque cuenta *tareas* pendientes, no *trabajo* pendiente: una imagen de cultivo cuesta en promedio ocho veces más que una lectura, y cuando una imagen del último lote le toca al trabajador lento, nadie se la puede sacar. El robo de trabajo sí puede, porque el ladrón se lleva tareas que todavía no empezaron.

Figura 5

Tiempo medio de espera en el clúster real

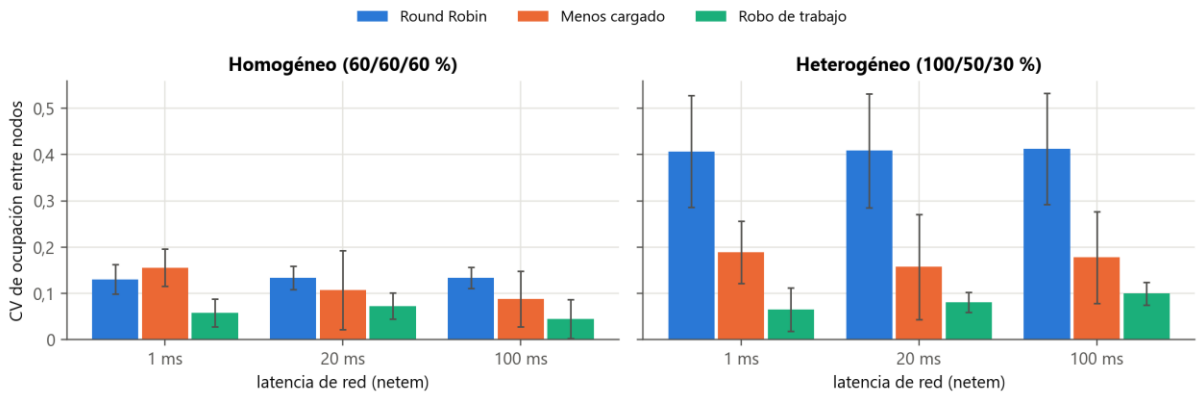


Nota. Espera = tiempo que la tarea pasó sin ejecutarse (colas y red).

Tiempo de espera. La espera media (Figura 5) separa las estrategias incluso donde el makespan no lo hace. En H0 con 1 ms, el robo de trabajo reduce la espera de 1,60 s (Round Robin) a 0,90 s, un 44 % menos, porque un trabajador que se queda sin cola toma tareas de los que tienen varias esperando. En H1 la diferencia es mayor: 4,87 s con Round Robin, 2,36 s con Menos cargado y 1,26 s con Robo de trabajo.

Figura 6

Desbalance de carga en el clúster real (CV del tiempo de ocupación)



Desbalance y mensajes. El coeficiente de variación del tiempo de ocupación (Figura 6) resume lo anterior: en H1 es 0,41 con Round Robin, entre 0,16 y 0,19 con Menos cargado y entre 0,07 y 0,10 con Robo de trabajo. El precio del robo de trabajo son los mensajes de control: además de las 96 confirmaciones que envían todas las estrategias, intercambia entre 125 y 300 mensajes de robo (pedidos y respuestas) por corrida, la mayoría fallidos en los momentos en que todos están ociosos entre lotes. Son mensajes de unos 160 bytes que viajan entre trabajadores, sin pasar por el coordinador.

Efecto de la heterogeneidad y de la latencia

Heterogeneidad. Es el factor que más pesa. Con la misma capacidad total, pasar de H0 a H1 aumenta el makespan de Round Robin un 44 % y el de Menos cargado un 9 %, mientras que el del robo de trabajo apenas cambia (1 a 4 %). Una estrategia estática es

aceptable cuando los nodos son iguales y se vuelve la peor opción cuando no lo son, que es exactamente el escenario de la institución: computadoras nuevas y viejas.

Latencia. Su efecto sobre el makespan es pequeño (unos pocos segundos sobre 40) porque los mensajes se superponen con el cómputo, pero se nota en la espera: de 1 a 100 ms, la espera media del robo de trabajo sube de 0,90 a 1,98 s en H0 y de 1,26 a 2,53 s en H1. El robo de trabajo es la estrategia más sensible a la latencia, porque cada robo cuesta una ida y vuelta entre trabajadores (unos 200 ms o más con 100 ms de netem) durante la cual el ladrón no ejecuta nada; por eso, con 100 ms, su ventaja sobre Menos cargado se reduce y los mensajes de robo bajan de unos 300 a unos 160 por corrida en H0 y de 250 a 125 en H1. Aun así, en H1 sigue siendo la mejor estrategia con los tres niveles de latencia.

Validez del simulador

La Tabla 13 compara, para cada una de las 18 configuraciones, la media de las 5 corridas reales con la media de las 5 simulaciones con las mismas semillas.

Tabla 13

Validación: makespan y espera media, reales y simulados, con su error relativo

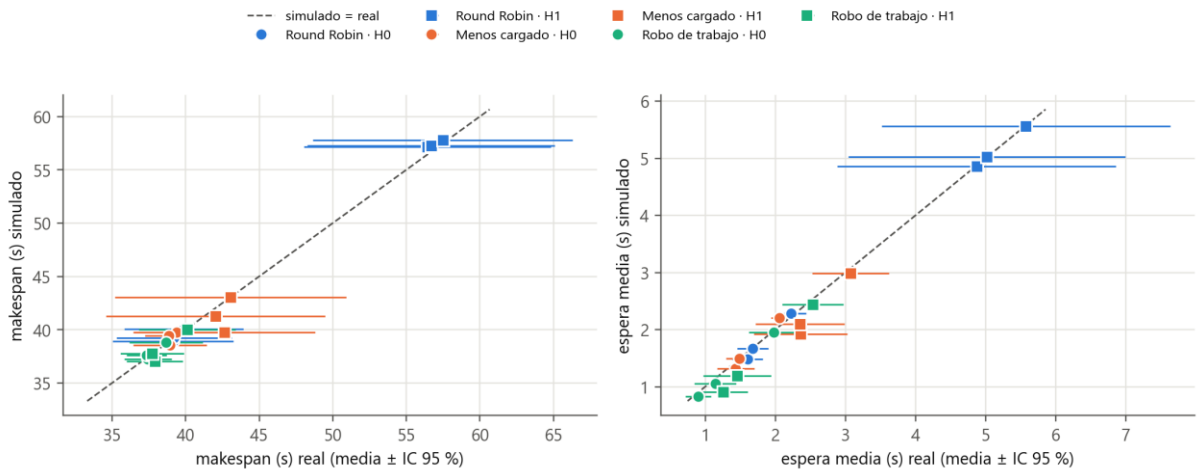
Het.	Lat.	Estrategia	Makesp. real	Makesp. sim.	Error (%)	Espera real	Espera sim.	Error (%)
H0	1	Menos cargado	39,38	39,74	0,9	1,43	1,32	8,0
H0	1	Round Robin	39,15	38,91	0,6	1,60	1,48	7,4
H0	1	Robo de trabajo	37,43	37,25	0,5	0,90	0,83	7,5
H0	20	Menos cargado	38,94	38,54	1,0	1,49	1,49	0,1
H0	20	Round Robin	39,21	39,21	0,0	1,68	1,67	0,5
H0	20	Robo de trabajo	37,34	37,59	0,7	1,14	1,05	7,6
H0	100	Menos cargado	38,85	39,46	1,6	2,05	2,21	7,5
H0	100	Round Robin	39,87	40,05	0,5	2,22	2,28	2,6
H0	100	Robo de trabajo	38,67	38,83	0,4	1,98	1,95	1,4
H1	1	Menos cargado	42,64	39,74	6,8	2,36	1,92	18,6
H1	1	Round Robin	56,44	57,13	1,2	4,87	4,86	0,2
H1	1	Robo de trabajo	37,90	37,01	2,4	1,26	0,90	28,1
H1	20	Menos cargado	42,03	41,25	1,9	2,35	2,09	11,1
H1	20	Round Robin	56,69	57,29	1,1	5,02	5,03	0,2
H1	20	Robo de trabajo	37,72	37,74	0,1	1,46	1,19	18,1
H1	100	Menos cargado	43,05	43,02	0,1	3,07	2,99	2,7

Het.	Lat.	Estrategia	Makesp. real	Makesp. sim.	Error (%)	Espera real	Espera sim.	Error (%)
H1	100	Round Robin	57,48	57,77	0,5	5,58	5,57	0,2
H1	100	Robo de trabajo	40,10	40,01	0,2	2,53	2,44	3,8

Nota. Valores en segundos. Real: media de 5 corridas; simulado: media de 5 simulaciones con las mismas semillas. Error = |simulado − real| / real.

Figura 7

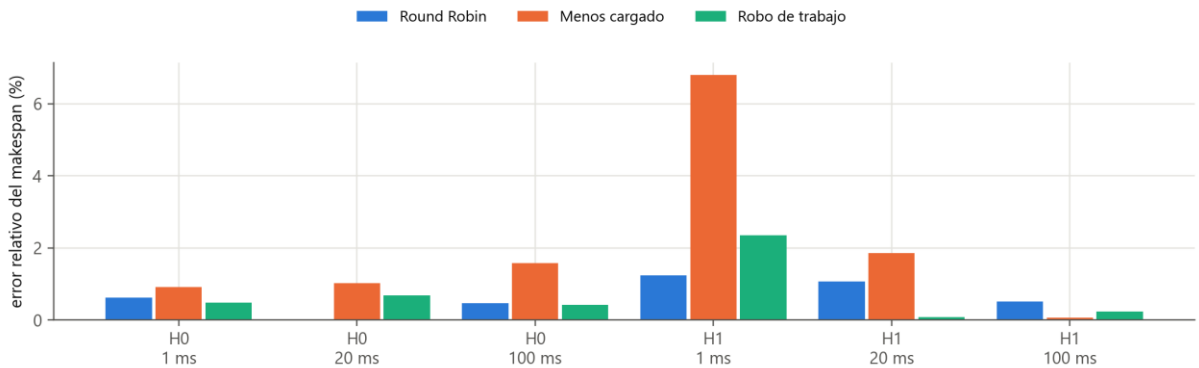
Valores simulados frente a reales



Nota. Cada punto es una configuración; círculos, H0; cuadrados, H1. La línea punteada marca simulado = real; las barras horizontales, el IC 95 % de las corridas reales.

Figura 8

Error relativo del makespan simulado por configuración



Makespan. El simulador acierta muy bien: el error relativo medio es del 1,1 % (mediana 0,6 %) y el máximo, del 6,8 % (Figuras 7 y 8). En las 18 configuraciones, el makespan simulado cae dentro del intervalo de confianza del 95 % de las corridas reales. El error más grande aparece en Menos cargado con H1 y 1 ms, que también es la configuración real con más variabilidad entre repeticiones.

Espera media y desbalance. El simulador es menos preciso en las métricas de segundo orden. La espera media tiene un error medio del 7 % y se subestima hasta un 28 %, sobre todo en las estrategias dinámicas con H1 y latencia baja, y el CV de ocupación, que es un número chico y muy variable, difiere en promedio un 20 % y hasta un 77 %. Es decir: el modelo reproduce bien cuánto tarda el sistema en total y peor cómo se reparte el tiempo dentro de él.

Lo que importa para decidir: el orden de las estrategias. En las seis condiciones, el simulador ordena las estrategias igual que el clúster real (Tabla 14): en todas, la mejor es la misma.

Tabla 14

Orden de las estrategias por makespan en el clúster real y en la simulación

Het.	Lat. (ms)	Orden real (de mejor a peor)	Orden simulado	τ de Kendall
H0	1	Robo de trabajo < Round Robin < Menos cargado	Robo de trabajo < Round Robin < Menos cargado	1,00
H0	20	Robo de trabajo < Menos cargado < Round Robin	Robo de trabajo < Menos cargado < Round Robin	1,00
H0	100	Robo de trabajo < Menos cargado < Round Robin	Robo de trabajo < Menos cargado < Round Robin	1,00
H1	1	Robo de trabajo < Menos cargado < Round Robin	Robo de trabajo < Menos cargado < Round Robin	1,00
H1	20	Robo de trabajo < Menos cargado < Round Robin	Robo de trabajo < Menos cargado < Round Robin	1,00
H1	100	Robo de trabajo < Menos cargado < Round Robin	Robo de trabajo < Menos cargado < Round Robin	1,00

Fuentes de error. Identificamos cuatro. (a) **Costos de software que el modelo no tiene:** el trabajador real es Python con dos hilos; recibir, deserializar y encolar una tarea, o atender un pedido de robo mientras el otro hilo calcula, cuesta tiempo que el modelo considera cero. Eso explica que el simulador sea optimista justo en la espera de las estrategias dinámicas, que son las que más mensajes procesan. (b) **Simplificaciones del modelo de red:** TCP real arranca cada conexión inactiva con una ventana chica y la va agrandando; el modelo usa un rendimiento constante por conexión, ajustado con un R^2 de 0,91 a 0,96 (Figura

3): sobreestima el tiempo de mensajes medianos y subestima el de los grandes. (c) **Sobrecarga de la virtualización:** VirtualBox corre sobre la interfaz de Hyper-V; además de lo visto en el arranque, comprobamos que dentro de las VM la contabilidad de CPU por *ticks* del kernel pierde entre un 30 y un 60 % del tiempo, de modo que ni siquiera las herramientas de monitoreo del invitado son del todo confiables. (d) **Interferencia del anfitrión:** las VM comparten la computadora con el resto de los programas del usuario, que durante la campaña siguieron en uso. Que el error del makespan sea tan bajo a pesar de todo se debe a que el makespan depende sobre todo de la velocidad de cómputo, el parámetro mejor calibrado.

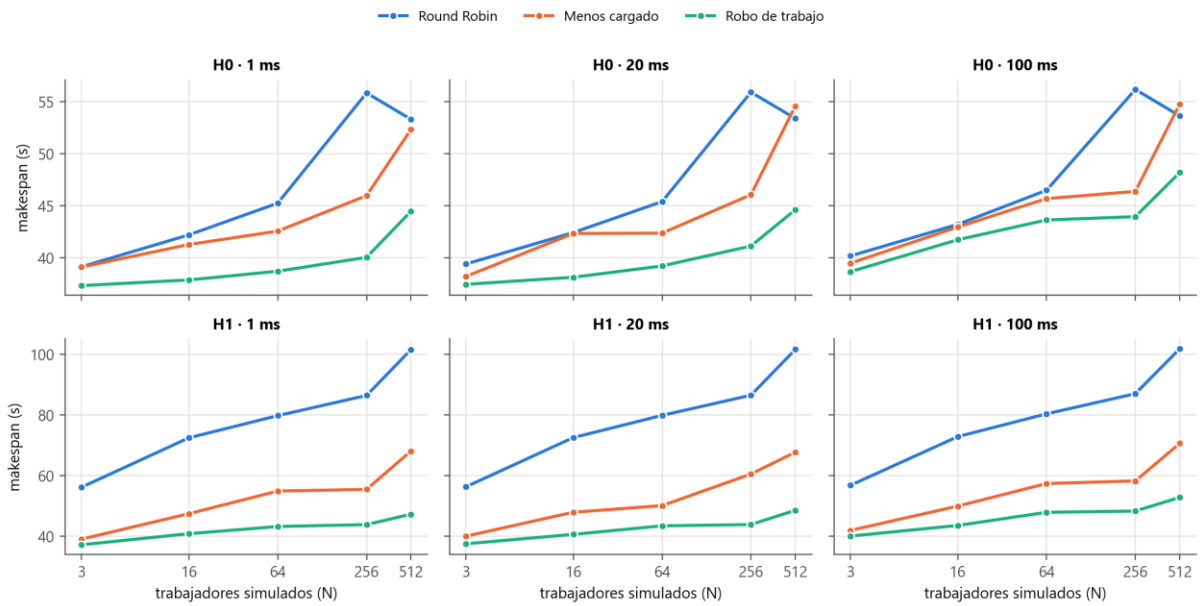
Escalabilidad

Con la plataforma calibrada simulamos 3, 16, 64, 256 y 512 trabajadores, con la carga creciendo en proporción a N (Figura 9). Tres resultados se repiten en las seis condiciones de latencia y heterogeneidad.

El orden de las estrategias se mantiene en todas las escalas. Robo de trabajo es siempre la más rápida y Round Robin la más lenta (Tabla 15). La recomendación que surge del clúster de tres nodos, entonces, no cambia al extrapolarla, lo que le da más peso.

Figura 9

Makespan simulado en función de la cantidad de trabajadores



Nota. La carga crece en proporción a N (tareas por lote = 8N/3), de modo que la utilización nominal es la misma en todas las escalas. Media de 3 semillas.

Tabla 15

Orden de las estrategias por makespan simulado en cada escala (de mejor a peor)

Het.	Lat. (ms)	N = 3	N = 16	N = 64	N = 256	N = 512
H0	1	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR
H0	20	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<RR<MC
H0	100	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<RR<MC
H1	1	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR
H1	20	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR
H1	100	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR	RT<MC<RR

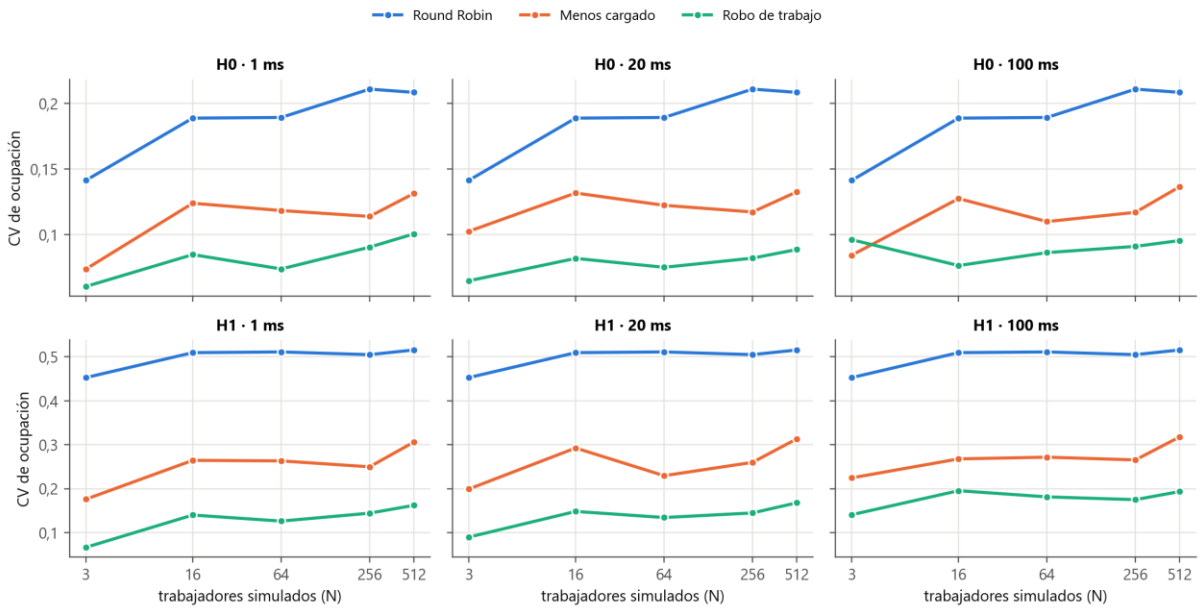
Nota. RT = Robo de trabajo; MC = Menos cargado; RR = Round Robin. Media de 3 semillas.

Las estrategias que no reaccionan empeoran con la escala. En H1, el makespan de Round Robin pasa de 56 s con 3 trabajadores a 73 s con 16, 80 s con 64, 87 s con 256 y 102 s con 512, aunque cada trabajador recibe en promedio la misma carga. Es un efecto de los valores extremos: con más trabajadores lentos es más probable que alguno reciba varias imágenes seguidas, y el makespan es el del peor. Menos cargado sigue la misma tendencia, atenuada (de 39 a 68 s). El robo de trabajo, en cambio, apenas pasa de 37 a 47 s, porque las

tareas que se acumulan en un nodo lento terminan en manos de los nodos libres. Lo mismo muestra el desbalance (Figura 10).

Figura 10

Desbalance simulado en función de la cantidad de trabajadores

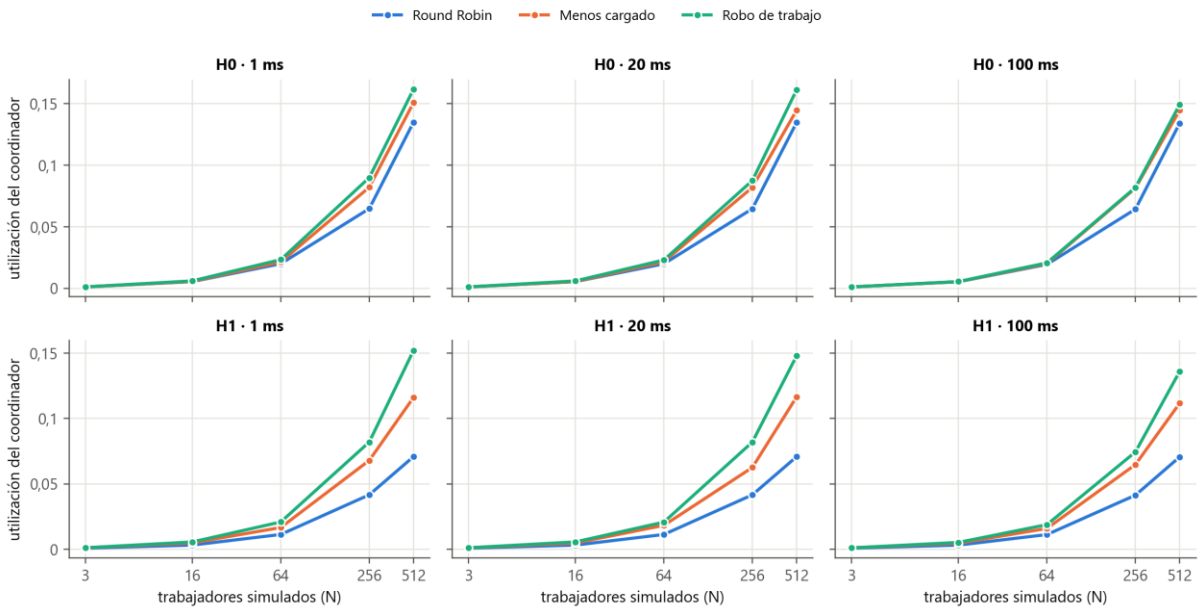


El límite del coordinador es su enlace de red, no su CPU. Entre 256 y 512 trabajadores el makespan de las tres estrategias salta a la vez, entre 5 y 15 s, y la espera media se duplica, mientras que la CPU del coordinador sigue ocupada solo entre un 7 y un 16 % del tiempo (Figura 11). La causa es que todas las tareas salen del coordinador con sus datos: cada lote de $8N/3$ tareas lleva en promedio 76 KiB por tarea, y el enlace del coordinador tiene una capacidad medida de 34 MB/s. La cuenta da que la demanda alcanza esa capacidad con unos 490 trabajadores, y la CPU del coordinador, a 2 mensajes de 219 µs por tarea, recién se saturaría con unos 2560. Las simulaciones adicionales con 384, 640 y 768 trabajadores (20 ms, una semilla) confirman el codo (Figura 12): con 384 trabajadores el robo de trabajo termina en 41,8 s en H0, con 512 en 44,6 s, y con 640 y 768 salta a 61,0 y 70,1 s; Round Robin y Menos cargado pasan de unos 53 s con 512 a más de 70 s con 640 y más de 84 s con 768. La CPU del coordinador, mientras tanto, no supera el 16 % de ocupación en ningún caso. El robo de trabajo sigue siendo la mejor estrategia después del codo, porque los robos

mueven datos entre trabajadores sin pasar por el coordinador, pero ninguna estrategia de reparto compensa un enlace saturado.

Figura 11

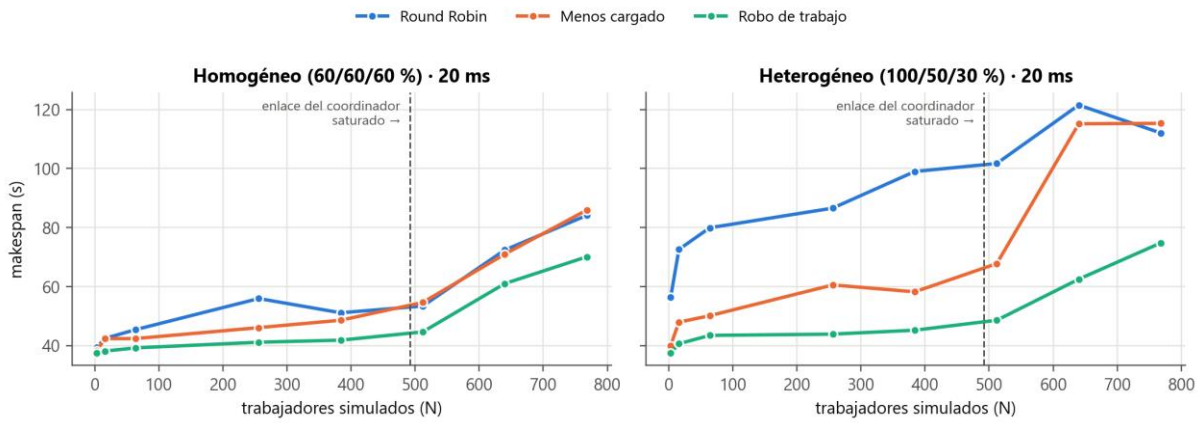
Utilización de la CPU del coordinador en la simulación



Nota. Fracción del makespan en que el núcleo del coordinador estuvo ocupado despachando tareas y procesando confirmaciones.

Figura 12

Makespan simulado alrededor del límite del enlace del coordinador (20 ms)



Nota. La línea punteada marca la escala a la que la demanda de datos de cada lote iguala la capacidad medida del enlace del coordinador (34 MB/s). Los puntos de 384, 640 y 768 trabajadores son de una sola semilla.

¿Hasta qué escala confiar en esta recomendación? El orden de las estrategias se apoya en dos cosas que el modelo representa bien, la velocidad de cómputo de cada nodo y la

propia lógica de las estrategias, que es el mismo código que corrió en el clúster; por eso lo consideramos confiable en todo el rango simulado. Los valores absolutos más allá de unos cientos de nodos dependen, en cambio, de la parte menos validada del modelo, la red: el codo de los 490 trabajadores sale de una capacidad medida en una red virtual de tres máquinas y de un modelo de TCP simplificado. Lo leemos como una advertencia de diseño (en esta arquitectura el coordinador es un cuello de botella de red antes que de cómputo) y no como un número para dimensionar equipos.

Consumo de recursos

Durante la campaña, el proceso de cada trabajador usó en promedio entre 0,33 y 0,41 núcleos (con picos de 0,8 en el trabajador más rápido), unos 180 MB de memoria por VM y menos de 1,2 Mbit/s de red; el coordinador ocupó casi 1,2 GB de sus 1,5 GB, la mayor parte en Prometheus y Grafana, y su proceso de coordinación prácticamente no usó CPU. En el anfitrión, los procesos de VirtualBox consumieron en promedio 1,4 núcleos de los 12 hilos disponibles y 3,3 GB de RAM, y el uso total de CPU de la computadora fue del 17 % en promedio, con picos del 48 % cuando el resto de los programas del usuario estaba activo (Tabla 16 y repositorio, `analysis/tablas/`).

La diferencia de costo entre emular y simular es la mayor de todo el trabajo (Tabla 16). La campaña real llevó 69 minutos de reloj (incluidas las tres corridas repetidas), sin contar el montaje del clúster ni los 12 minutos de calibración; las 90 simulaciones equivalentes tardaron 20 segundos en total, incluido el arranque de un proceso por simulación, con menos de 4 segundos de simulación propiamente dicha. La extrapolación completa, que en el mundo real requeriría cientos de computadoras, llevó 37 minutos en una sola VM con una vCPU; el costo de cada simulación crece más rápido que N (0,2 s con 16 trabajadores, 6,6 s con 256 y 31 s con 512), porque cada trabajador es un hilo del sistema

operativo y el núcleo de SimGrid resuelve el reparto del ancho de banda entre todas las comunicaciones activas.

Tabla 16

Costo en tiempo de emular frente a simular

Qué	Tiempo de reloj (min)
Campaña real (90 corridas, incluye cambios de condiciones)	69,4
Suma de corridas reales	63,8
Validación simulada (90 simulaciones)	0,1
Extrapolación simulada (CPU acumulada)	35,2

Ventajas y desafíos

Lo que dio la infraestructura como código. Tener las máquinas, su configuración y las condiciones experimentales en archivos versionados cambió la forma de trabajar: cada problema de montaje se resolvió una vez, en el archivo correspondiente, y quedó resuelto para todo el grupo y para cualquier tercero. Las condiciones de cada bloque (netem y CPUQuota) se aplican con el mismo *playbook* que usa el protocolo automático, de modo que no hay pasos manuales que alguien pueda olvidar entre una corrida y otra. Que el coordinador verifique el *hash* del código de los trabajadores antes de cada corrida evitó medir con versiones distintas en los dos extremos.

Lo que costó la reproducibilidad. Los tres problemas del paso 1 tienen algo en común: ninguno es un error del laboratorio, sino una interacción con el entorno del anfitrión (una VM huérfana de otra materia, Hyper-V activo por WSL2 y Docker, los permisos de las carpetas compartidas en Windows). Son justamente los que un `README.md` escrito desde la computadora de quien desarrolló no anticipa, y por eso la prueba en una computadora que no participó del desarrollo (paso 8) es indispensable. También aprendimos a fijar versiones y a evitar dependencias implícitas: el paquete de Ansible de Ubuntu instala por recomendación cientos de megabytes de colecciones que no usamos, y el módulo que agrega el repositorio de Grafana necesita una biblioteca de Python que no viene en la imagen base.

Diferencias entre plataformas. En Linux, Vagrant puede usar libvirt/KVM directamente y es la combinación más rápida; además, Ansible puede ejecutarse desde el anfitrión. En Windows hay que elegir entre VirtualBox y Hyper-V y, si Hyper-V está activo por otras herramientas, VirtualBox corre más lento y, como vimos, con restricciones en la cantidad de vCPU. En macOS con procesadores ARM (Apple Silicon) la box `bento/ubuntu-24.04` para amd64 no sirve: hace falta una box arm64 y un proveedor que la soporte (VMware Fusion, Parallels o QEMU), y SimGrid y el resto del laboratorio funcionarían igual porque no dependen de la arquitectura. Lo que sí cambiaría son las velocidades medidas, por lo que la calibración habría que repetirla en cada anfitrión.

Optimizaciones posibles. (a) Construir una box propia con Packer que ya traiga los paquetes, para que `vagrant up` no dependa de descargas y tarde una fracción del tiempo; (b) validar automáticamente el `Vagrantfile` y los *playbooks* en integración continua (por ejemplo, `ansible-lint` y una ejecución con contenedores en GitHub Actions); (c) reemplazar las VM de los trabajadores por contenedores en el experimento de escala intermedia (decenas de nodos), aceptando que comparten el núcleo del anfitrión; (d) mover Prometheus y Grafana a una VM propia o limitarles la memoria: con el tablero refrescándose cada 5 s, Grafana 13 llegó a ocupar 0,9 GB de los 1,5 GB del coordinador en dos horas y lo dejó sin memoria hasta que lo reiniciamos; y (e) sacar los datos de las tareas del camino del coordinador, que es lo que la extrapolación muestra como límite a gran escala.

Conclusión

Hallazgos sobre planificación y balanceo de carga

El resultado principal es que la heterogeneidad, y no la latencia, es lo que decide la estrategia. Con nodos iguales, cualquier reparto razonable da el mismo tiempo total y la diferencia entre estrategias queda en el tiempo de espera de cada tarea. Con nodos desiguales y la misma capacidad total, una política estática como Round Robin pierde un 44 % de

tiempo porque el nodo más lento se convierte en el cuello de botella, mientras que el robo de trabajo absorbe la heterogeneidad casi por completo. Menos cargado queda en el medio por una razón concreta: decide con la cantidad de tareas pendientes, no con el trabajo que representan, y una vez que una tarea pesada llegó a un nodo lento ya no se mueve.

La latencia afecta poco al makespan, porque los mensajes se superponen con el cómputo, pero encarece justamente a la estrategia que mejor funciona: cada robo es una ida y vuelta entre trabajadores, y con 100 ms de retardo la ventaja del robo de trabajo se achica. A gran escala, la extrapolación indica que el diseño centralizado no se agota por la CPU del coordinador sino por su enlace de red, por el que pasan los datos de todas las tareas: con 20 ms de latencia, el makespan del robo de trabajo pasa de 44,6 s con 512 trabajadores a 61 s con 640 y 70 s con 768, mientras la CPU del coordinador nunca supera el 16 % de ocupación; la cuenta ubica el límite en unos 490 trabajadores.

Para la institución del escenario, la recomendación es usar robo de trabajo, separar el camino de los datos del camino de las decisiones (que los trabajadores reciban del coordinador solo la referencia a cada lote y busquen los datos en otro lado) y no confiar en un único coordinador más allá de unos pocos cientos de nodos sin antes volver a validar el modelo a esa escala.

Respuesta a la pregunta guía

¿Cuánto se puede confiar en una simulación para tomar decisiones de diseño de un sistema distribuido? Según lo que medimos, bastante, con tres condiciones. La primera es que esté **calibrada** con mediciones del sistema real: nuestro primer modelo de red, que tomaba el rendimiento de una conexión TCP como capacidad del enlace, habría anticipado un cuello de botella que no existe. La segunda es que esté **validada** en el rango donde se puede medir: en nuestras 18 configuraciones el makespan simulado se apartó del real un 1,1 % en promedio y el orden de las estrategias fue siempre el mismo, pero la espera media y el desbalance se

apartaron bastante más. La tercera es **usarla para lo que validó**: el simulador es confiable para comparar estrategias y para encontrar dónde aparece un cuello de botella, y mucho menos para predecir valores absolutos de métricas finas o el comportamiento de partes del sistema que el modelo simplifica, como la pila TCP o los costos de software de cada mensaje. Cuanto más lejos de la escala validada, más conviene leer sus predicciones como tendencias y no como números.

Reflexión de cada integrante desde su rol

Salazar, Facundo (Investigador/a de modelado y simulación). Desde el modelado aprendí que lo difícil de simular no es escribir los actores sino decidir qué detalle del sistema real importa. Que el modelo de SimGrid importe el mismo módulo de estrategias que usa el clúster eliminó una fuente entera de discrepancias. La extrapolación me enseñó a separar decidir de predecir: el orden de las estrategias se mantuvo en todas las escalas, pero los valores absolutos lejos de la escala validada hay que leerlos como tendencias. Levantar el clúster con Vagrant me mostró que un laboratorio definido como código depende también del entorno donde corre: los dos problemas que más tiempo costaron, la VM huérfana registrada en VirtualBox y el cuelgue con 2 vCPU sobre Hyper-V, no estaban en nuestros archivos sino en la computadora anfitriona. Calibrar en pares con Romina me hizo ver cuánto depende el modelo de la calidad de las mediciones.

Quintana, Sofía (Analista de experimentación y datos). Desde la experimentación aprendí que el diseño (factorial completo, repeticiones, mismas semillas y orden aleatorio) es lo que permite afirmar que una diferencia se debe a la estrategia y no al azar o al entorno. También aprendí que la reproducibilidad se cuida en los detalles: tres corridas se repitieron solo porque cambió la versión del código registrada, y una prueba en la misma computadora no alcanza para decir que el laboratorio es reproducible. Escribir el README pensando en

alguien que no conoce el proyecto me obligó a entender la arquitectura completa, no solo mi parte.

Vera, Romina (Ingeniero/a de infraestructura como código). Desde mi rol aprendí que la idempotencia hay que comprobarla: describir un estado en lugar de escribir comandos es lo que permite correr el playbook las veces que haga falta. Programar el coordinador y los trabajadores me mostró de cerca problemas propios de los sistemas distribuidos: no se pueden comparar relojes de máquinas distintas y un mensaje puede perderse en silencio si el middleware no avisa. La calibración fue lo que más me enseñó: el ancho de banda que se mide con mucha latencia no es la capacidad de la red sino lo que rinde una conexión TCP, y usarlo mal habría dado una predicción falsa a gran escala. Validar el simulador me dejó claro hasta dónde se le puede creer: acierta el makespan y el orden de las estrategias, pero no la espera. Trabajar en pares con Facundo me ayudó a entender cómo el modelo de SimGrid usa las mediciones.

Propuestas para trabajos futuros

- **Fallas de nodos en SimGrid.** SimGrid permite apagar y encender *hosts* durante la simulación. Con eso se podría estudiar qué estrategia tolera mejor la caída de un trabajador: en el robo de trabajo, las tareas en la cola del nodo caído se pierden, salvo que el coordinador las reenvíe al no recibir su confirmación.
- **Migración de procesos.** Las tres estrategias mueven tareas que todavía no empezaron. Permitir migrar una tarea en ejecución, con su estado parcial, cambiaría la política de selección y su costo (Sinha, 1997).
- **Kubernetes como planificador.** Reemplazar el coordinador por un clúster de Kubernetes y comparar su planificador, que ubica *pods* según los recursos pedidos, con las estrategias propias, usando la misma carga.

- **Datos fuera del coordinador.** Repetir la extrapolación con los datos servidos por un almacenamiento distribuido y medir hasta dónde se corre el límite del coordinador.
- **Uso del laboratorio en otras asignaturas.** Como todo se levanta con `vagrant up`, el mismo laboratorio sirve para prácticas de sincronización de relojes, exclusión mutua o tolerancia a fallas, cambiando solo el código de la aplicación.

Referencias

Blumofe, R. D., & Leiserson, C. E. (1999). Scheduling multithreaded computations by work stealing. *Journal of the ACM, 46*(5), 720–748. https://doi.org/10.1145/324133.324234

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM, 59*(5), 50–57. https://doi.org/10.1145/2890784

Casanova, H., Giersch, A., Legrand, A., Quinson, M., & Suter, F. (2014). Versatile, scalable, and accurate simulation of distributed applications and platforms. *Journal of Parallel and Distributed Computing, 74*(10), 2899–2917. https://doi.org/10.1016/j.jpdc.2014.06.008

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed systems: Concepts and design* (5.ª ed.). Addison-Wesley.

Eager, D. L., Lazowska, E. D., & Zahorjan, J. (1986). Adaptive load sharing in homogeneous distributed systems. *IEEE Transactions on Software Engineering, SE-12*(5), 662–675. https://doi.org/10.1109/TSE.1986.6312961

Geerling, J. (2020). *Ansible for DevOps: Server and configuration management for humans* (2.ª ed.). Leanpub.

Grafana Labs. (s.f.). *Grafana documentation*. Recuperado el 21 de octubre de 2026, de https://grafana.com/docs/grafana/latest/

HashiCorp. (s.f.). *Vagrant documentation*. Recuperado el 21 de octubre de 2026, de https://developer.hashicorp.com/vagrant/docs

HashiCorp. (2023, 10 de agosto). *HashiCorp adopts Business Source License* [Entrada de blog]. https://www.hashicorp.com/blog/hashicorp-adopts-business-source-license

Hashimoto, M. (2013). *Vagrant: Up and running*. O'Reilly Media.

Hemminger, S. (2005). Network emulation with NetEm. En *Proceedings of the 6th Australia's National Linux Conference (LCA 2005)*. Linux Australia.

Hintjens, P. (2013). *ZeroMQ: Messaging for many applications*. O'Reilly Media.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Jain, R. (1991). *The art of computer systems performance analysis: Techniques for experimental design, measurement, simulation, and modeling*. Wiley.

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter Development Team. (2016). Jupyter Notebooks: A publishing format for reproducible computational workflows. En F. Loizides & B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

Legrand, A., Marchal, L., & Casanova, H. (2003). Scheduling distributed applications: The SimGrid simulation framework. En *Proceedings of the 3rd IEEE/ACM International Symposium on Cluster Computing and the Grid (CCGrid 2003)* (pp. 138–145). IEEE. https://doi.org/10.1109/CCGRID.2003.1199362

McKinney, W. (2010). Data structures for statistical computing in Python. En *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.

Mitzenmacher, M. (2001). The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems, 12*(10), 1094–1104. https://doi.org/10.1109/71.963420

Morris, K. (2020). *Infrastructure as code: Dynamic systems for the cloud age* (2.ª ed.). O'Reilly Media.

Ousterhout, K., Wendell, P., Zaharia, M., & Stoica, I. (2013). Sparrow: Distributed, low latency scheduling. En *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13)* (pp. 69–84). ACM. https://doi.org/10.1145/2517349.2522716

Prometheus Authors. (s.f.). *Prometheus documentation*. Recuperado el 21 de octubre de 2026, de https://prometheus.io/docs/

Red Hat. (s.f.). *Ansible documentation*. Recuperado el 21 de octubre de 2026, de https://docs.ansible.com/

Shivaratri, N. G., Krueger, P., & Singhal, M. (1992). Load distributing for locally distributed systems. *Computer, 25*(12), 33–44. https://doi.org/10.1109/2.179115

SimGrid Team. (s.f.). *SimGrid documentation*. Recuperado el 21 de octubre de 2026, de https://simgrid.org/doc/latest/

Singhal, M., & Shivaratri, N. G. (1994). *Advanced concepts in operating systems*. McGraw-Hill.

Sinha, P. K. (1997). *Distributed operating systems: Concepts and design*. IEEE Press.

Tanenbaum, A. S., & van Steen, M. (2023). *Distributed systems* (4.ª ed.). distributed-systems.net.

Velho, P., Schnorr, L. M., Casanova, H., & Legrand, A. (2013). On the validity of flow-level TCP network models for grid and cloud simulations. *ACM Transactions on Modeling and Computer Simulation, 23*(4), Artículo 23. https://doi.org/10.1145/2517448

Anexo A: bitácoras individuales

Cada integrante redactó su bitácora desde su rol. Las versiones vivas están en `docs/bitacoras/` del repositorio.

Bitácora individual - Salazar, Facundo

Rol: Investigador/a de modelado y simulación (Avanzado)

Entregables propios: pasos 1, 4 y 7 (Vagrantfile multimáquina, modelo SimGrid y plataformas XML, extrapolación a gran escala); además, video de demostración y presentación de la defensa.

21/10/2026 - Paso 1: clúster como código con Vagrant (en pares con Romina)

Qué hice

- Escribí el Vagrantfile: los trabajadores salen de la variable N_WORKERS (3 por defecto) y la misma definición genera el inventario de Ansible.
- Con Romina hicimos la primera puesta en marcha alternando quién escribía: vagrant up, vagrant ssh, vagrant halt, vagrant destroy -f y un nuevo vagrant up sin intervención.

Qué aprendí

- Que definir máquinas e inventario en un único lugar evita que se desincronicen cuando cambia la cantidad de trabajadores.

Dificultades que encontré

- vagrant up falló con E_ACCESSDENIED: VirtualBox tenía registrada una VM vieja cuyos archivos ya no existían, y Vagrant consulta los puertos de todas las VMs registradas.
- Las VMs quedaban colgadas en "Loading essential drivers". El anfitrión tiene Hyper-V activo y VirtualBox corre encima; con 2 vCPU el arranque se colgaba en forma intermitente.

Cómo las resolví

- Desregistramos la VM huérfana (con copia de seguridad del registro de VirtualBox).
- Probé por separado chipset, paravirtualización y cantidad de vCPU; solo 1 vCPU arrancó siempre, así que todas las VMs quedaron con 1 vCPU y la heterogeneidad se controla con CPUQuota.

21/10/2026 - Paso 4: modelo en SimGrid

Qué hice

- Escribí el modelo de SimGrid (S4U, Python; src/agrolab/sim.py) con la misma estructura de actores que el sistema real: un coordinador y un actor por trabajador, que importan el mismo módulo de estrategias (strategies.py) que usa el clúster.
- Escribí el generador de plataformas XML en estrella (src/agrolab/platform.py): un enlace para el coordinador y uno por trabajador.
- Verifiqué el modelo con una tarea y un nodo (agrolab simulate trivial): el tiempo simulado coincide con la cuenta a mano, con una diferencia relativa del orden de 10^{-7}.

Qué aprendí

- Que para que simulación y realidad ejecuten "el mismo algoritmo" lo más seguro es que importen el mismo código, no reescribirlo dos veces.

Dificultades que encontré

- Modifiqué archivos del simulador mientras Sofía corría la campaña real, y la columna con el hash del código cambió en tres corridas aunque el código medido era el mismo.

Cómo las resolví

- Aparté mis cambios hasta el final de la campaña y Sofía repitió esas corridas, para que todo el conjunto de datos tuviera la misma versión.

21/10/2026 - Paso 6: calibración (en pares con Romina)

Qué hice

- Acompañé a Romina en agrolab calibrate, alternando quién escribía, y en llevar los valores medidos a las plataformas XML del modelo.

21/10/2026 - Paso 7: extrapolación a gran escala

Qué hice

- Extrapolé de 3 a 512 trabajadores (270 simulaciones, agrolab simulate scale), con la carga creciendo en proporción a los trabajadores (8N/3 tareas por lote) para mantener la utilización nominal del clúster real.
- Agregué 384, 640 y 768 trabajadores con 20 ms para ubicar el codo: el límite del coordinador es su enlace de red, cerca de 490 trabajadores; su CPU nunca pasó del 16 %.

Qué aprendí

- Que el simulador sirve para decidir (el orden de las estrategias no cambió en ninguna escala) pero no para dar números finos: la espera y el desbalance son las métricas donde más se nota lo que el modelo deja afuera.

Dificultades que encontré

- Grafana, con el tablero refrescándose cada 5 s, llegó a ocupar casi 1 GB del coordinador en dos horas y frenó las simulaciones.

Cómo las resolví

- Reinicié Grafana y no dejé el tablero abierto durante las simulaciones largas.

21/10/2026 - Video y presentación de la defensa

Qué hice

- Grabé el video de demostración (5:10, docs/video/): vagrant up desde un clon nuevo, idempotencia, apagar y encender, una corrida real con Grafana, la misma corrida simulada y los resultados.
- Armé la presentación de la defensa (13 diapositivas), con una parte para los pasos de cada integrante.

Reflexión final desde el rol

Desde el modelado aprendí que lo difícil de simular no es escribir los actores sino decidir qué detalle del sistema real importa. Que el modelo de SimGrid importe el mismo módulo de estrategias que usa el clúster eliminó una fuente entera de discrepancias. La extrapolación me enseñó a separar decidir de predecir: el orden de las estrategias se mantuvo en todas las escalas, pero los valores absolutos lejos de la escala validada hay que leerlos como tendencias. Levantar el clúster con Vagrant me mostró que un laboratorio definido como código depende también del entorno donde corre: los dos problemas que más tiempo costaron, la VM huérfana registrada en VirtualBox y el cuelgue con 2 vCPU sobre Hyper-V, no estaban en nuestros archivos sino en la computadora anfitriona. Calibrar en pares con Romina me hizo ver cuánto depende el modelo de la calidad de las mediciones.

Bitácora individual - Quintana, Sofía

Rol: Analista de experimentación y datos (Inicial)

Entregables propios: pasos 0, 5 y 8 (diagnóstico y acta de roles, campaña en el clúster real, prueba de reproducibilidad); además, cuaderno Jupyter y README de reproducción.

21/10 y 25/10/2026 - Paso 0: diagnóstico y acta de roles

Qué hice

- Organicé la autoevaluación de los tres integrantes y redacté el acta de roles (docs/acta-de-roles.md).
- Relevé qué computadoras del grupo podían ejecutar VirtualBox y Vagrant; Facundo creó el repositorio en su cuenta de GitHub.

Qué aprendí

- Que la autoevaluación sirve para repartir el trabajo según lo que cada uno ya sabe hacer y lo que le conviene aprender.

Dificultades que encontré

- Romina y Facundo quedaron en el mismo nivel (Avanzado).

Cómo las resolví

- Aplicamos la regla de la consigna: los roles entre ellos se asignaron por las dimensiones de mayor puntaje.

21/10/2026 - Paso 5: experimentación en el clúster real

Qué hice

- Generé la planilla de corridas (agrolab plan) desde experiments/params.toml: 3 estrategias × 2 niveles de heterogeneidad × 3 latencias (1, 20 y 100 ms), 5 repeticiones, en orden aleatorio y con las mismas semillas en todas las configuraciones.
- Ejecuté la campaña completa (agrolab run), con la supervisión de Romina: 90 corridas en 66 minutos, sin fallas de comunicación ni corridas trabadas.
- Exporté el consumo de las VMs desde Prometheus (analysis/export_prometheus.py).

Qué aprendí

- Que usar las mismas semillas en todas las configuraciones (números aleatorios comunes) y aleatorizar el orden evita confundir el efecto de la estrategia con la carga sorteada o con una deriva del anfitrión (Jain, 1991).

Dificultades que encontré

- Tres corridas del primer bloque quedaron registradas con otro hash de código porque se modificaron archivos del simulador mientras se ejecutaban.

Cómo las resolví

- Repetí esas tres corridas al final de la campaña (36,41, 37,18 y 38,20 s contra 36,36, 37,18 y 38,27 s de las originales) y conservé las filas originales en descartadas_runs.csv y descartadas_tasks.csv.

21/10, 25/10 y 26/10/2026 - Paso 8: prueba de reproducibilidad

Qué hice

- 21/10, prueba previa en la computadora de desarrollo: destruimos el clúster, clonamos el repositorio desde GitHub en una carpeta nueva y seguimos solo el README. El vagrant up terminó en 18,4 minutos sin intervención; la segunda ejecución de Ansible dio changed=0; la corrida de prueba dio 40,41 s contra 40,50 s de la campaña original, y 40,03 s en SimGrid.
- La prueba en otra computadora la hicimos juntas con Romina el 26/10, en su notebook, que no participó del desarrollo; no se usó mi computadora porque tiene 8 GB de RAM pero no le queda memoria libre suficiente para las cuatro VM (3,2 GB) más el sistema. Se siguió solo el README y cada paso quedó registrado en docs/prueba-reproducibilidad.md. El vagrant up desde un clon nuevo tardó 11,1 minutos, la idempotencia dio changed=0, Grafana y Prometheus respondieron y el ciclo de apagado y encendido funcionó.

- Con la misma semilla, las corridas reales dieron 49,7 s (Robo de trabajo), 52,0 s (Menos cargado) y 73,6 s (Round Robin), contra 40,5, 43,9 y 60,9 s de la campaña: el mismo orden, unos 20 % más lentas.

Qué aprendí

- Que una prueba en la misma computadora no alcanza: esa máquina ya tenía la imagen base descargada y los problemas de su entorno resueltos.
- Que el laboratorio se reproduce, pero los tiempos dependen del anfitrión: la calibración del simulador vale para la computadora donde se hizo.

Dificultades que encontré

- Un primer intento se hizo con la notebook a batería y dio tiempos casi el doble de largos, porque Windows baja la frecuencia de la CPU.
- vagrant destroy dejó carpetas vacías en VirtualBox VMs y el siguiente vagrant up falló con VERR_ALREADY_EXISTS.

Cómo las resolví

- Se descartó ese intento y se repitió la prueba desde cero con la notebook reiniciada y cargando.
- Se borraron las carpetas sobrantes y las dos fallas quedaron en los problemas conocidos del README.

21/10 y 26/10/2026 - Cuaderno de análisis y README

Qué hice

- Armé el cuaderno analysis/analisis.ipynb: tablas resumen con media, desvío e intervalo de confianza, ANOVA y figuras del clúster real, de la validación y de la extrapolación.
- Redacté el README de reproducción paso a paso, con los problemas conocidos.

Reflexión final desde el rol

Desde la experimentación aprendí que el diseño (factorial completo, repeticiones, mismas semillas y orden aleatorio) es lo que permite afirmar que una diferencia se debe a la estrategia y no al azar o al entorno. También aprendí que la reproducibilidad se cuida en los detalles: tres corridas se repitieron solo porque cambió la versión del código registrada, y una prueba en la misma computadora no alcanza para decir que el laboratorio es reproducible. Escribir el README pensando en alguien que no conoce el proyecto me obligó a entender la arquitectura completa, no solo mi parte.

Bitácora individual - Vera, Romina

Rol: Ingeniero/a de infraestructura como código (Avanzado)

Entregables propios: pasos 2, 3 y 6 (Ansible, netem, CPUQuota, Prometheus y Grafana; módulo común de estrategias, coordinador y trabajadores en Python; calibración y validación del simulador); además, supervisión del paso 5 y compilación del informe.

21/10/2026 - Paso 1: primera puesta en marcha (en pares con Facundo)

Qué hice

- Levanté el clúster con Facundo alternando quién escribía y anoté los comandos y los problemas que aparecieron, que después pasamos en los problemas conocidos del README.

21/10/2026 - Paso 2: aprovisionamiento con Ansible

Qué hice

- Escribí provisioning/site.yml y los cuatro roles (common, worker, coordinator, monitoring), que se ejecutan desde el coordinador con ansible_local.
- Escribí conditions.yml y los scripts lab-netem y lab-cpuquota, que aplican la latencia y la heterogeneidad de cada bloque.

- Configuré Prometheus y el tablero de Grafana.
- Corrí el playbook dos veces (vagrant provision coord --provision-with ansible): la segunda dio changed=0 en los cuatro nodos.
- Supervisé las corridas de Sofía en el paso 5, sobre todo la aplicación de netem y CPUQuota en cada bloque.

Qué aprendí

- Que la idempotencia hay que comprobarla: describir el estado deseado en lugar de los comandos es lo que permite correr el playbook las veces que haga falta.

Dificultades que encontré

- Windows no puede ser nodo de control de Ansible.

Cómo las resolví

- Usé ansible_local: Ansible se instala dentro del coordinador y desde ahí configura a los trabajadores, sin instalar nada en el anfitrión.

21/10/2026 - Paso 3: implementación de las estrategias

Qué hice

- Escribí el paquete agrolab: generador de carga con semilla, las tres estrategias en un módulo común (strategies.py), el protocolo de mensajes, el trabajador (dos hilos: comunicaciones y ejecutor) y el coordinador (un hilo con bucle de eventos).
- Elegimos ZeroMQ en lugar de RabbitMQ: con VMs de 512 MB un broker de Erlang compite por memoria y, para el robo de trabajo, los trabajadores tienen que hablar entre sí sin pasar por el coordinador.
- Probé coordinador y trabajadores en localhost antes de tener las VMs (tests/smoke_local.py) y escribí pruebas del módulo común (tests/test_strategies.py).

Qué aprendí

- Que en ZeroMQ el ROUTER descarta en silencio los mensajes a destinos desconocidos salvo que se active ROUTER_MANDATORY; activarlo hizo que los errores de conexión aparezcan.
- Que para no comparar relojes de máquinas distintas conviene medir todo con el reloj del coordinador y que el trabajador informe solo duraciones.

Dificultades que encontré

- La prueba local falló porque la función que arma los mensajes tenía un parámetro kind y el mensaje de tarea también tenía un campo kind.

Cómo las resolví

- Hice el primer parámetro de pack() solo posicional.

21/10/2026 - Paso 6: calibración y validación (en pares con Facundo)

Qué hice

- Medí la velocidad de cada VM con la tarea de referencia (unas 53 000 unidades/s sin límite; con CPUQuota 60 % da 33 000, con 50 % 28 000 y con 30 % 16 000) y el tiempo de ida de mensajes de 0 a 256 KiB con 0, 1, 20 y 100 ms de netem (agrolab calibrate).
- Simulé las 18 configuraciones del clúster con las mismas semillas (agrolab simulate validate): el makespan simulado se apartó del real un 1,1 % en promedio (máximo 6,8 %) y el orden de las estrategias fue el mismo en las 6 condiciones. La espera media se apartó hasta un 28 %, porque el modelo no tiene los costos de Python de cada mensaje.

Qué aprendí

- Que el ancho de banda "efectivo" que se mide con mucha latencia (0,47 MB/s con 100 ms) no es la capacidad de la red: es lo que rinde una conexión TCP con su

ventana. Multiplicando rendimiento por ida y vuelta da una ventana casi constante (100 a 140 KiB) en los cuatro niveles.

Dificultades que encontré

- Si se usaba ese valor como capacidad de los enlaces, el simulador predecía que a gran escala el enlace del coordinador se saturaba con muy poco tráfico.

Cómo las resolví

- Con Facundo cargamos en el modelo la capacidad medida sin netem para el enlace del coordinador y el rendimiento de una conexión para el de cada trabajador.

26/10/2026 - Paso 8: prueba de reproducibilidad en mi notebook (con Sofía)

Qué hice

- Puse mi notebook, que no participó del desarrollo, para la prueba de reproducibilidad que tenía a cargo Sofía, y la hicimos juntas siguiendo solo el README.
- Después de la prueba agregué al informe los resultados y las dos fallas encontradas.

21/10 al 26/10/2026 - Informe

Qué hice

- Compilé el informe en formato APA con las secciones de cada integrante y revisé las referencias.

Reflexión final desde el rol

Desde mi rol aprendí que la idempotencia hay que comprobarla: describir un estado en lugar de escribir comandos es lo que permite correr el playbook las veces que haga falta. Programar el coordinador y los trabajadores me mostró de cerca problemas propios de los sistemas distribuidos: no se pueden comparar relojes de máquinas distintas y un mensaje

puede perderse en silencio si el middleware no avisa. La calibración fue lo que más me enseñó: el ancho de banda que se mide con mucha latencia no es la capacidad de la red sino lo que rinde una conexión TCP, y usarlo mal habría dado una predicción falsa a gran escala. Validar el simulador me dejó claro hasta dónde se le puede creer: acierta el makespan y el orden de las estrategias, pero no la espera. Trabajar en pares con Facundo me ayudó a entender cómo el modelo de SimGrid usa las mediciones.

Anexo B: acta de roles

Cátedra: Sistemas Operativos Distribuidos · Ingeniería en Sistemas de Información · FITA

Docentes: Jara, Juan Emiliano · Ing. Quiñones, Paola

Ciclo lectivo: 2026 · Comisión "A" · Sede Posadas

1. Autoevaluación de experiencia

Escala por dimensión: 0 = nunca lo usé · 1 = lo usé con guía · 2 = lo uso con cierta autonomía · 3 = lo uso con autonomía y puedo explicarlo a otros.

Dimensión	**Quintana, Sofía**	**Salazar, Facundo**	**Vera, Romina**
Linux y línea de comandos	1	3	2
Programación (Python/C/JS, concurrencia, sockets)	1	3	3
Redes (TCP/IP, puertos, NAT, DNS, diagnóstico)	1	2	2
Virtualización y contenedores	1	3	3
Conceptos de SOD (Ejes 1 y 2)	1	2	2
Total (/15)	5	**13**	12
Nivel	Inicial	Avanzado	Avanzado

Referencia: 0–5 Inicial · 6–10 Intermedio · 11–15 Avanzado.

Cada integrante completa su columna de forma individual. Si dos integrantes quedan en el mismo nivel, los roles se asignan por las dimensiones de mayor puntaje (fortalezas complementarias).

2. Asignación de roles

Rol	**Nivel**	**Integrante**	**Entregables propios**
Investigador/a de modelado y simulación	Avanzado	Salazar, Facundo	Pasos 1, 4 y 7: Vagrantfile multimáquina, modelo SimGrid (S4U Python) y plataformas XML, extrapolación a gran escala. Además, video de demostración y presentación de la defensa

Ingeniero/a de infraestructura como código	Avanzado	Vera, Romina	Pasos 2, 3 y 6: playbooks y roles de Ansible, emulación de red con netem y CPUQuota, Prometheus y tablero de Grafana, módulo común de estrategias, coordinador y trabajadores en Python, calibración y validación del simulador. Además, supervisión del paso 5 y compilación del informe
Analista de experimentación y datos	Inicial	Quintana, Sofía	Pasos 0, 5 y 8: diagnóstico y acta de roles, planilla de corridas y campaña real, prueba de reproducibilidad en otra computadora. Además, cuaderno Jupyter y README de reproducción

3. Pasos críticos en pares

Paso	**Integrantes**	**Modalidad**
Paso 1 — primera puesta en marcha del clúster con Vagrant	Salazar (Avanzado) + Vera (Avanzado)	misma terminal o pantalla compartida, alternando quién escribe
Paso 6 — calibración del simulador	Salazar (Avanzado) + Vera (Avanzado)	ídem

4. Reglas acordadas

- Cada rol es responsable de sus entregables, pero todo el grupo debe poder explicar la arquitectura completa en la defensa.
- Cada integrante registra su bitácora semanal en `docs/bitacoras/`.
- Todo cambio de código, configuración o datos pasa por el repositorio.

5. Conformidad

Integrante	**Firma**	**Fecha**
Quintana, Sofía	Sofía Quintana	26/10/2026
Salazar, Facundo	Facundo Salazar	26/10/2026
Vera, Romina	Romina Vera	26/10/2026

Validación del equipo docente: _____ Fecha: ____/____/2026

Anexo C: planilla de corridas

La planilla completa (90 corridas) está en `experiments/planilla.csv`. Se reproduce el primer bloque como ejemplo del formato que siguió el rol de experimentación.

Tabla 17

Primer bloque de la planilla de corridas (L1-H0)

Orden	Corrida	Latencia (ms)	Heterog.	Estrategia	Rep.	Semilla
1	L1-H0-rr-r1	1	H0	Round Robin	1	1000
2	L1-H0-ll-r5	1	H0	Menos cargado	5	1004
3	L1-H0-rr-r3	1	H0	Round Robin	3	1002
4	L1-H0-ws-r2	1	H0	Robo de trabajo	2	1001
5	L1-H0-ws-r4	1	H0	Robo de trabajo	4	1003
6	L1-H0-rr-r5	1	H0	Round Robin	5	1004
7	L1-H0-ll-r3	1	H0	Menos cargado	3	1002
8	L1-H0-ll-r2	1	H0	Menos cargado	2	1001
9	L1-H0-rr-r4	1	H0	Round Robin	4	1003
10	L1-H0-ws-r5	1	H0	Robo de trabajo	5	1004
11	L1-H0-ws-r1	1	H0	Robo de trabajo	1	1000
12	L1-H0-ws-r3	1	H0	Robo de trabajo	3	1002
13	L1-H0-ll-r4	1	H0	Menos cargado	4	1003
14	L1-H0-ll-r1	1	H0	Menos cargado	1	1000
15	L1-H0-rr-r2	1	H0	Round Robin	2	1001

Nota. El orden dentro del bloque se sorteó con la semilla 2026 (params.toml, design.order_seed).

Anexo D: archivos de plataforma y comandos de reproducción

Plataforma SimGrid calibrada para el nivel H1 con 20 ms de netem (generada por `agrolab.platform`):

```
<?xml version='1.0'?>
<!DOCTYPE platform SYSTEM "https://simgrid.org/simgrid.dtd">
<platform version="4.1">
  <zone id="agrolab" routing="Dijkstra">
    <host id="coord" speed="52476.828f"/>
    <host id="w1" speed="54153.421f"/>
    <host id="w2" speed="27834.772f"/>
    <host id="w3" speed="16220.153f"/>
    <router id="switch"/>
    <link id="l_coord" bandwidth="34078324Bps" latency="0.019178474s"/>
    <link id="l_w1" bandwidth="1620308Bps" latency="0.019178474s"/>
    <link id="l_w2" bandwidth="1620308Bps" latency="0.019178474s"/>
    <link id="l_w3" bandwidth="1620308Bps" latency="0.019178474s"/>
    <route src="coord" dst="switch"><link_ctn id="l_coord"/></route>
    <route src="w1" dst="switch"><link_ctn id="l_w1"/></route>
    <route src="w2" dst="switch"><link_ctn id="l_w2"/></route>
    <route src="w3" dst="switch"><link_ctn id="l_w3"/></route>
  </zone>
</platform>
```

Comandos para reproducir el trabajo desde cero:

```
git clone https://github.com/RominaaVeraa/tif-sod-vagrant-simgrid
cd tif-sod-vagrant-simgrid
vagrant up
vagrant ssh coord
agrolab calibrate
agrolab simulate trivial
agrolab plan && agrolab run
agrolab simulate validate
agrolab simulate scale
```